AOSP 进阶丛书 · 第九册 · ANDROID 13 (API 33)

后台与电源五系统合流

JobScheduler 十二控制器与配额计费、Alarm 的政策化定价、Doze 七态与维护窗、Standby 五桶惩罚、Power 唤醒度与 PowerGroup、Thermal 三层治理——后台经济学的完整规则书。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目录

前言：后台的"经济学"——时间、配额与瓦特 2
第一章 全景：五套系统的分工与合流 2
第二章 Doze：整机节流的总闸 3
第三章 JobScheduler：条件与配额的调度器 5
第四章 AlarmManager：唯一的准点源 8
第五章 App Standby：按"不常用"逐级惩罚 8
第六章 PowerManagerService：醒与睡的治理 9
第七章 Thermal：性能的反面货币 11
第八章 设计决策十问 12
附录 A 观测实操 13
附录 B 快问快答（16 条） 13
附录 C 自检题 14
附录 D 动手实验室（一天量） 14
第九章 QuotaController 深读：配额窗口、桶预算与触发限流 14
第十章 WakeLock 的一生：从 acquire 到内核唤醒源 20
第十一章 BatteryService：电量数据的来路与六路下游 24
第十二章 灭屏全链：从一次超时到内核挂起 28
教科书习题集（分级） 32
术语表 36
排障手册：后台调度与电源问题定位 37
附录 E：后台调度与电源核心机制深度速查表 40

前言：后台的"经济学"——时间、配额与瓦特

前八册讲的都是"前台世界"（怎么启动、怎么画、怎么通信）。本册进入**后台世界**：谁能在用户看不见时继续干活、干多久、用什么代价——这套"后台经济学"由五套系统合谋：JobScheduler（条件与配额）、AlarmManager（准点）、Doze（整机节流）、App Standby（按应用惩罚）、Power/Thermal（醒睡与降速的物理约束）。

先记一条路径事实（读码第一坑）：本树（Android 13）里 JobScheduler/Alarm/Doze/AppStandby 的源码在 `frameworks/base/apex/jobscheduler/service/java/com/android/server/`（下文 JS=…/apex/jobscheduler/service/java/com/android/server）——它们就是 Mainline 模块 com.android.scheduling 的实体（书1 §16.3 表的活证据）；而 PowerManagerService/ThermalManagerService 仍在 `services/core/.../power/`。旧教材的 `services/core/.../job|alarm` 路径在本树**不存在**。

本册还有一个隐藏主线：**每一层都在把"开发者的意志"翻译成"系统的时序"**——Job 把条件翻译成窗口、Doze 把静止翻译成深睡、Standby 把冷落翻译成惩罚、Thermal 把温度翻译成降速。读完全册你会获得一种后台问题的通用分析法：先问"这件事系统想让我什么时候干"，再写代码——顺序反了就永远在对抗。

体例不变：真实源码逐行注释+调用链+图。读码路线：全景（一章）→ Doze（二章）→ Job（三章）→ Alarm（四章）→ Standby（五章）→ Power（六章）→ Thermal（七章）。

第一章 全景：五套系统的分工与合流

> **设计洞察 · 学习目标**
>
> 1. 说出五套系统分工的"合流三句话"；2. 记住本树 Job/Alarm/Doze 的新路径（apex/jobscheduler）与旧教材路径陷阱；3. 建立"先问系统何时让我干"的分析顺序；4. 用一次"每天 3 点同步"对比两种写法的功耗量级



图 1 后台与电源全景：Job/Alarm/Doze/Standby 四调节器+Power/Thermal 两物理约束，含本树新路径提示

合流三句话（面试级浓缩）：**Alarm 管准点、Job 管条件，两者都先过"桶"与"Doze"两道闸；Power 决定整机醒睡、Thermal 决定降速降亮**。一次"每天 3 点同步"的正确姿势是 JobScheduler+约束声明（桶友好），而不是精确闹钟+前台服务保活——两种写法功耗差一个量级。

第二章 Doze：整机节流的总闸

设计洞察 · 学习目标

1. 默画 Doze 七态流转与 QUICK_DOZE 旁路；2. 说清 SENSING"廉价最后一问"的对冲设计；3. 列出深睡三限制与三层白名单账本；4. 用 dumpsys deviceidle 观察状态机现场

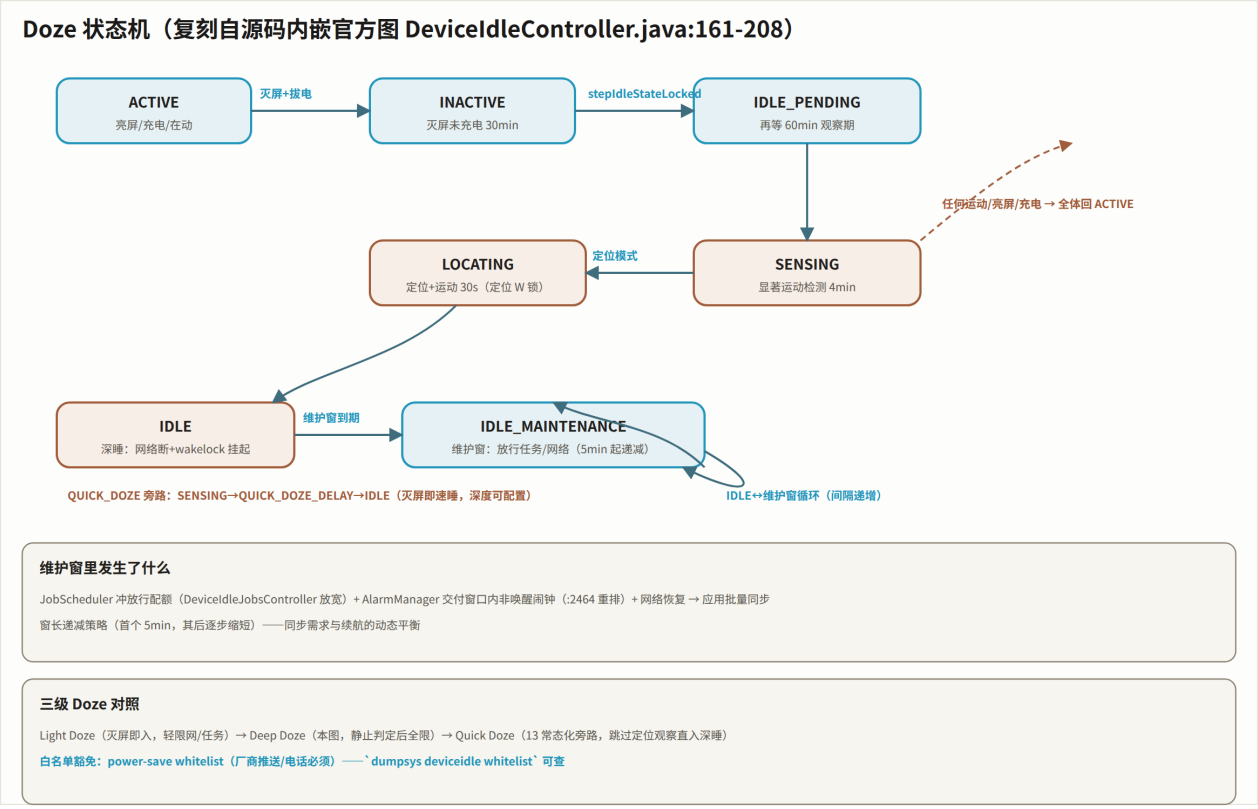


图 2 Doze 状态机（复刻自源码内嵌官方图 DeviceIdleController.java:161-208）：七态流转与 QUICK_DOZE 旁路

2.1 状态机逐段精读（真实源码骨架）

java JS/DeviceIdleController.java（骨架+讲解注释）

```
// 讲解注释①：入口三刺激：灭屏拔电（→INACTIVE，30min 计时）、
// 亮屏/充电/运动（→ACTIVE，becomeActiveIfAppropriateLocked）。
// 讲解注释②：stepIdleStateLocked 是唯一推进器（官方 DOT 图每条边都标它）：
// INACTIVE→IDLE_PENDING（再等 60min）→SENSING（显著运动检测 4min，
// 有动即回 ACTIVE——口袋里走路不睡）→LOCATING（30s 定位观察，拿 WAKE_LOCATE）
// →IDLE（深睡：网络断、wakelock 挂起、Job 冻结）
// 讲解注释③：IDLE↔IDLE_MAINTENANCE 循环：维护窗放行同步（首个约 5min，
// 之后递减）——"深睡但定期开窗"是 Doze 的核心妥协。
// 讲解注释④：QUICK_DOZE 旁路（:200 附近）：SENSING 后跳过 LOCATING
// 直入深睡——13 的默认轻量化路径（灭屏即睡）。
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释后台调度与电源的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

设计洞察 · 逐段精读：SENSING 为什么存在

深睡代价大（断网冻结），误判代价也大（用户正拿起手机却断网）。SENSING 用加速度计"显著运动"做**廉价的最后一问**——4 分钟无显著动才敢继续。这是"节能收益 vs 误判成本"的经典对冲，与书5 按键拦截链的"先问再分发"同构。

2.2 深睡里到底限了什么

三张清单：网络（除白名单全断）、wakelock（PARTIAL 也挂起，仅白名单例外）、Job/Alarm（非唤醒闹钟延到维护窗 `adjustDeliveryTimeBasedOnDeviceIdle:2464` 批量重排）。白名单三档：厂商预置（推送/电话）、用户加的、临时豁免（`dumpsys deviceidle` 全景）。

2.3 白名单与豁免的三层结构

text DeviceIdleController 的豁免账本（读码归纳）

```
mPowerSaveWhitelistAll // 全量白名单（厂商+用户+系统）
mPowerSaveWhitelistExceptIdle // 只豁免 Doze 不豁免省电模式（临时豁免常驻于此）
mPowerSaveWhitelistApps // 用户可见/可管理的子集（电池优化页）
// 讲解注释：Doze 中可用的=ExceptIdle+临时豁免；推送 SDK 靠厂商预置或
// 用户自弃电池优化进门——这就是"电池优化"开关的源码实体。
```

第三章 JobScheduler：条件与配额的调度器

设计洞察 · 学习目标

1. 描述 Job 从 schedule 到 onStartJob 的四站；2. 背出十二控制器的"与闸"结构与典型卡点；3. 解释 QuotaController"排队不拒绝"的哲学；4. 理解 bind 模式中系统的生杀权与 WorkManager 的库级定位



图 3 Job 调度管线：入账→十二控制器"与"闸→Quota 计费→bind 执行，含 WorkManager 关系

3.1 提交与执行链逐段（真实锚点）

java JS/job/JobSchedulerService.java + JobServiceContext（骨架+讲解注释）

```
// 讲解注释①：JobInfo.Builder 只声明约束（网络类型/充电/空闲/最小延迟…），
// scheduleAsPackage(:1104) 入账 JobStatus；JobStore 落盘（重启不丢）。
// 讲解注释②：maybeQueueReadyJobsForExecutionLocked(:2406)：从就绪池挑队
// ——"就绪"= 十二控制器全部 satisfied（controllers/ 目录：
// Connectivity/Battery/Idle/Storage/ContentObserver/…/Quota/Restricting）。
// 讲解注释③：执行走 bindService！JobServiceContext(:79) implements
// ServiceConnection → handleServiceBoundLocked(:693/763) → onStartJob。
// 讲解注释④：bind 的意义：系统握生杀权——约束失效/超时即 unbind 收编；
// jobFinished 只是"报告完成"，不是"保活"。
```

3.2 QuotaController：执行层计费员（13 核心）

滚动窗口（10 分钟/24 小时）对每 App 记"执行时长+次数"账：前台态免费、后台按 Standby 桶打折（ACTIVE 足额 → RESTRICTED 近零）。超配额不拒绝、只排队——**把"能不能跑"变成"何时轮到"**。`dumpsys jobscheduler` 的 Job quotas 段即账本现场。WorkManager 是这层的库级封装（链式依赖/加急通道 expedited），系统视角只有 Job——"WorkManager 保活"是伪命题，保的是重试语义。

3.2.1 十二控制器速查（controllers/ 目录）

控制器	监听约束	典型卡点
ConnectivityController	网型（UNMETERED/NOT_ROAMING …）	连了计费网
BatteryController	充电中/电量阈值	拔线即取消执行
IdleController	设备空闲（Doze 联动）	深睡中反而不跑（等维护窗）
ContentObserverController	URI 变更触发	内容没变
StorageController	存储不紧张	低存储
BackgroundJobsController	App 后台资格	桶/省电剥夺
ComponentController	组件启用态	Receiver 被禁用
DeviceIdleJobsController	维护窗放宽（Doze 专属豁免）	窗内才放
PrefetchController	预取预测（13）	非预测时段
QuotaController	执行配额	账本见 §3.3
RestrictingController	桶限制	RARE 以下
TimeController	deadline 到点	最小延迟未到

读法：**每个控制器=一个"订阅了某信号的闸"**；`dumpsys jobscheduler` 的 job 详情逐控制器给出 STOPPED-REASON——排障即查表。

3.3 QuotaController 逐段（:887 isWithinQuotaLocked）

java JS/job/controllers/QuotaController.java:887 起（骨架+讲解注释）

```
boolean isWithinQuotaLocked(@NonNull final JobStatus jobStatus) {
// 讲解注释①：三重豁免：前台/充电中/Doze 维护窗内 → 免检直过
//  （:613 的调用点在"就绪→可执行"最后一闸）。
// 讲解注释②：否则按 (userId, packageName, standbyBucket) 查两本账：
// 10 分钟窗口（短时配额）与 24 小时窗口（长时配额），
// 执行时长与次数双维计费；桶越深赊账额度越低。
// 讲解注释③：超账 → 返回 false：Job 留在就绪池，等窗口滚动释放额度
// ——不是失败，是"下一班再走"。
}
```

逐段精读②的账本设计：双窗口防止两种博弈——纯短窗（攒爆发）与纯长窗（饿死近期活跃者）。**执行计费而不是排队计费**：真正跑了的才算钱，被约束卡住的不扣——鼓励把 Job 写成"快速幂等"而不是长跑。

3.4 老伙计：SyncManager 与 ContentResolver.sync

ContentResolver 的同步生态最终也汇入 Job（13 已迁 JobScheduler 驱动）：account+authority 的周期同步声明在 SyncManager 翻译成带约束的 Job。读码一景：

onPerformSync 跑在框架的同步进程/你的 Service 里，超时与退避由 sync 框架托管——**"同步"是 Job 之上的一层领域语言**，与 WorkManager 同构（都是"把领域需求翻译成系统调度原语"）。

第四章 AlarmManager：唯一的准点源

设计洞察 · 学习目标

1. 区分 RTC/ELAPSED 与 wakeup 与否的四象限；2. 解释闹钟合并与"唤醒源数不等于闹钟数"；3. 说出 AllowWhileIdle 的 9 分钟配额及政策含义；4. 用"合并余地"视角选择精确/宽松 API

java JS/alarm/AlarmManagerService.java:2210 起（骨架+讲解注释）

```
// 讲解注释①：setImpl(:2210)：类型（RTC/ELAPSED、wakeup 与否）+
// 窗口（window）——精确 vs 宽松的本质是"系统有多少合并余地"。
// 讲解注释②：内核层已是 timerfd（/dev/alarm 退役）：批量近邻闹钟
// 合并唤醒（省电靠合并，准点靠 wakeup 型独走）。
// 讲解注释③：Doze 交互：非 wakeup 闹钟在深睡中全部延至维护窗
//  （adjustDeliveryTimeBasedOnDeviceIdle:2464）；AllowWhileIdle
// 每 App 每 9 分钟一次的"深睡豁免"配额——高频唤醒的天敌。
```

4.1 闹钟批量（Batch）与唤醒合并

text AlarmManagerService 的节能机制（读码归纳）

```
mAlarmStore 按触发时间组织（:2626 setImplLocked 入库）
• 非 wakeup 闹钟：OS 可任意延迟交（等下一次 CPU 醒）——合并的自然土壤
• wakeup 闹钟：必须准点唤醒 CPU——近邻（同 量级 窗口）合并唤醒源，
各自 handler 仍按点投递
• Doze 中：全量非 wakeup 延至维护窗（:2464 重排），豁免账本同 Doze 白名单
// 讲解注释：所以"耗电排行里的 alarm 唤醒数"是合并后的唤醒源数，
// 不是闹钟个数——优化目标是减"源"不是减"钟"。
```

设计洞察 · 逐段精读：setExact 和 setAndAllowWhileIdle 的社会成本

前者准点但阻止合并（全员准点=全员费电）；后者打穿 Doze 但有 9 分钟/App 配额。**框架用"配额"把开发者逐向宽松窗口与 Job 化**——API 设计即政策，这是读系统代码最该学的一课。

第五章 App Standby：按"不常用"逐级惩罚

设计洞察 · 学习目标

1. 背出五桶阈值与各自待遇；2. 描述判分两路（15 天 PAR 窗口+使用预测）与一步提桶路径；3. 掌握 am set-standby-bucket 验证法；4. 说明桶对 Job/Alarm/后台网络三个下游的影响



图 4 App Standby 五桶（ACTIVE→RESTRICTED）：判分两路、三个下游与观测实战

五桶语义（AppStandbyController.java）：ACTIVE(10) 免费一切 → WORKING_SET(20) Job 频率约半 → FREQUENT(30) 约 1/8 → RARE(40) 几乎无后台 → RESTRICTED(45) 每日窗口级+FGS 受限。判分两路：15 天 PAR 窗口的"多久没碰"+ 13 的使用模式预测模型；**用户主动点开=一步回 ACTIVE**（:726/:767 提桶路径）。三个下游：Job 频率（Restricting/Quota）、Alarm 窗口、后台网络。实战：`am set-standby-bucket pkg restricted` 是验证"受限行为"的标准手法。

第六章 PowerManagerService：醒与睡的治理

设计洞察 · 学习目标

1. 说出唤醒度四态与 PowerGroup 的按屏独立；2. 走读 userActivityNoUpdateLocked 的合法性三查与 NoUpdate 命名架构；3. 区分 WakeLock 汇总层与 SuspendBlocker 物理层；4. 说清省电模式"策略叠加器"与 Doze 的正交分工

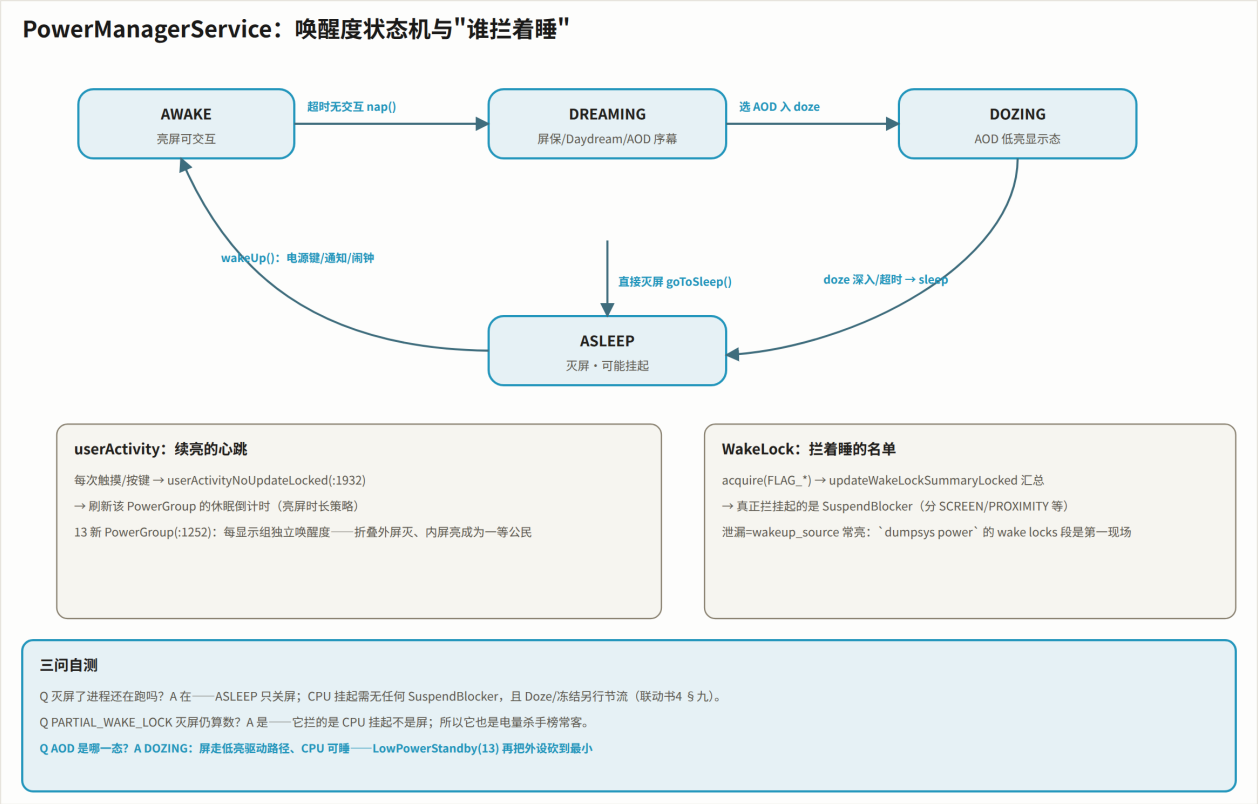


图 5 唤醒度状态机（AWAKE/DREAMING/DOZING/ASLEEP）与 PowerGroup、WakeLock→SuspendBlocker 名单

6.1 唤醒度逐段（真实锚点）

java services/core/.../power/PowerManagerService.java（骨架+讲解注释）

```
// 讲解注释①：四态 WAKEFULNESS_{AWAKE,DREAMING,DOZING,ASLEEP}
//  （import :28-31）；转移入口 wakeUpInternal/goToSleepInternal/
// napInternal，落点 setWakefulnessLocked → 广播（Notifier）+
// 显示/窗口联动（书5 的屏亮度与 doze 屏）。
// 讲解注释②：userActivityNoUpdateLocked(:1932)：触摸/按键续亮——
// 刷新该 PowerGroup 的休眠倒计时；13 新 PowerGroup(:1252)：
// 唤醒度按显示组独立（折叠双屏各自亮灭成为一等公民）。
// 讲解注释③：WakeLock 汇总（updateWakeLockSummaryLocked）→
// 真正拦 CPU 挂起的是 native SuspendBlocker（分屏/接近/常规）；
// "灭屏了进程还在跑"的真相：ASLEEP 只关屏，挂起需无人拦。
```

6.1.1 userActivity 真源码逐行（:1932-1966）

java services/core/.../power/PowerManagerService.java:1932 起（节选+讲解注释）

```
private boolean userActivityNoUpdateLocked(long eventTime, int event, int flags, int uid) {
// 讲解注释①：13 的关键变化——遍历全部 PowerGroup 分发：
// 一次触摸只续亮"事件所属显示组"，其他组不受牵连
//  （外接屏演示时手机屏可按自己的倒计时先睡）。
for (int idx = 0; idx < mPowerGroups.size(); idx++) {
if (userActivityNoUpdateLocked(mPowerGroups.valueAt(idx), ...)) ...
}
}
private boolean userActivityNoUpdateLocked(final PowerGroup powerGroup, ...) {
// 讲解注释②：合法性三查：事件时间不能早于上次睡/醒（防老事件回灌
```

```
// 把刚灭的屏又"续亮"）、systemReady 之前一律忽略。
if (eventTime < powerGroup.getLastSleepTimeLocked()
|| eventTime < powerGroup.getLastWakeTimeLocked() || !mSystemReady) {
return false;
}
// 讲解注释③：按 event 类型（TOUCH/KEY/BUTTON/OTHER）套用
// 不同的 dim/off 时长策略 → powerGroup.userActivity() 刷新倒计时。
...
// 讲解注释④：返回 needUpdate → 上层 updatePowerState() 一把梳全状态
// （唤醒度/亮度/SuspendBlocker 重算）——"记账归记账、结算一次过"。
}
```

设计洞察 · 逐段精读：为什么叫 NoUpdate

命名即架构：*NoUpdateLocked 只改账本（各 group 的倒计时），不立即触发任何状态机——统一由外层 updatePowerState 结算。**先记账后结算**让并发事件（触摸+按键同帧）合并成一次状态重算，是 PMS 全书反复出现的主模式。

6.2 LowPowerStandby（13 新）

`LowPowerStandbyController`（:71）：息屏待机低功耗态——AOD 之外的"极简醒着"（蓝牙/AOD 按配置关停），定位在"既要待机秒响应又要省电"的中间态。与 Doze 的分工：Doze 治"后台任务"，LowPowerStandby 治"息屏后的整机外设"。

6.3 省电模式（Battery Saver）家族

`power/batterysaver/` 子目录：省电模式是**策略叠加器**——不改唤醒度状态机，而是在各处消费点挂闸（亮度帽/AOD 关/CPU 频率档/动画降帧/后台限制加强），分档（off/trigger 5%/on）。与 Doze 正交：Doze 治"没人用时"，省电治"电量焦虑时"。自动触发规则（5%/15% 可配）在 `BatterySaverStateMachine`——面试常问"省电模式到底省在哪"，答案就是这张闸门清单。

第七章 Thermal：性能的反面货币

设计洞察 · 学习目标

1. 列出温度流与严重级的六档矩阵；2. 描述 HAL 直管频点/framework 联动/应用回调三层治理；3. 记住 :267 热关机兜底与第十一章 :402 电量关机的双闸分工；4. 用 cmd thermalservice override-status 复现热态

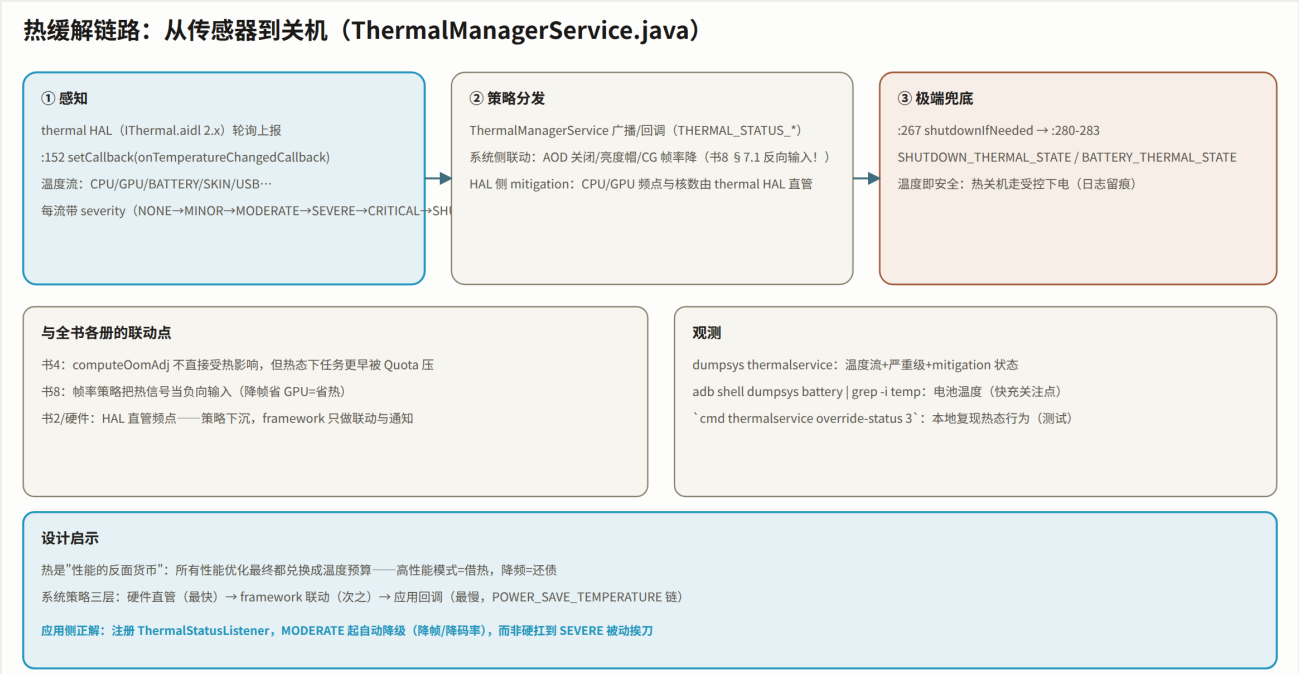


图 6 热缓解链路：HAL 感知（:152 回调）→策略联动→极端兜底（:267 关机），与全书联动点

温度流（CPU/GPU/BATTERY/SKIN…）× 严重级（NONE→SHUTDOWN 六档）。三层治理：**HAL 直管频点**（最快最底层）、framework 联动（AOD 关/亮度帽/帧率降——书8 § 7.1 的帧率策略把热当负向输入）、应用回调（THERMAL_STATUS_*，MODERATE 起自觉降级）。:267 `shutdownIfNeeded` 是终极兜底——热是安全议题。观测 `dumpsys thermalservice`；测试用 `cmd thermalservice override-status` 本地复现热态。

第八章 设计决策十问

- **为什么 Job 用 bind 而非 start?** 生杀权：约束失效即收编；start 出去的进程系统只能杀不能优雅停。
- **为什么配额是"排队"不是"拒绝"?** 后台任务是合法需求，拒绝会逼开发者对抗（保活回潮）；排队把冲突化为时序。
- **为什么 Doze 要维护窗?** 深睡不可持续（同步/推送会饿死），开窗递减是"续航与及时性"的动态帕累托。
- **为什么 SENSING 先于 IDLE?** 深睡误判代价高，加速度计是最便宜的二次确认。
- **为什么 AllowWhileIdle 限 9 分钟?** 打穿 Doze 的豁免必须稀缺，否则 Doze 名存实亡。
- **为什么五桶而不用连续分?** 离散档位可解释、可配置、可 UI 呈现（电池页"受限"）；连续分无谈判界面。
- **为什么 PowerGroup 按显示组?** 多屏时代的"这块屏的事这块屏管"，避免一屏拖全机。
- **为什么 wakelock 汇总再分层 SuspendBlocker?** 语义按用途（屏/接近/CPU），物理按是否拦挂起——两层各自演化。
- **为什么热治理 HAL 直管频点?** 热时间常数短，framework 转发太慢；策略下沉、联动上浮。

- **为什么这五套系统要分立而非合一？** 时间维度（Alarm）、条件维度（Job）、整机维度（Doze）、应用维度（Standby）、物理维度（Power/Thermal）——正交分治，各自可演进（Job/Alarm 已 Mainline 化而 Power 未必，即证据）。

附录 A 观测实操

目标	命令	看点
任务全景	dumpsys jobscheduler	约束位图/桶/配额/正在执行
闹钟全景	dumpsys alarm	每 App 闹钟数/类型/下次触发
Doze 状态	dumpsys deviceidle	当前态/白名单/维护窗统计
桶状态	am get-standby-bucket <pkg>	五桶之一
强制受限	am set-standby-bucket <pkg> restricted	行为验证
电源全景	dumpsys power	唤醒度/wakelocks/SuspendBlocker
热状态	dumpsys thermalservice	温度流+severity+mitigation

附录 B 快问快答（16 条）

- 本树 Job 源码在哪？——frameworks/base/apex/jobscheduler/（com.android.scheduling 模块实体）。
- Job 十二控制器逻辑是什么？——全部 satisfied 的"与"。
- JobService 与普通 Service 的执行差异？——bind 拉起、系统可收回。
- Quota 打折依据？——Standby 桶（前台免费）。
- WorkManager 与 JobScheduler？——库与系统服务；WM 链式/持久化，底层 Job/Alarm 双通道。
- Doze 深睡三限制？——网络断/wakelock 挂起/任务窗口化。
- 维护窗递减什么意思？——首个约 5min，后续逐次缩短（同步需求递减假设）。
- QUICK_DOZE 是什么？——跳过 LOCATING 的快速深睡旁路（13 常态）。
- 充电会退出 Doze 吗？——会（→ACTIVE）。
- AllowWhileIdle 配额？——每 App 每 9 分钟一次豁免。
- 五桶怎么快速升降？——用户点开即 ACTIVE；15 天不用渐进降桶。
- RESTRICTED 的 FGS 待遇？——受限（书4 §3.4 联动）。
- PARTIAL_WAKE_LOCK 灭屏有效吗？——有效，拦 CPU 挂起（电量杀手常客）。
- AOD 是哪个唤醒度？——DOZING（低亮驱动路径）。

- 热的 framework 动作有哪些？——AOD/亮度/帧率联动+应用回调；频点 HAL 直管。
- 学完往哪走？——把书4 冻结/书8 帧率与本册串成"功耗三件套"面试叙事。

附录 C 自检题

- 默画 Doze 七态与每条边的触发条件（含 QUICK 旁路）。
- 一个 Job 从 schedule 到 onStartJob 经过哪四站？bind 的生杀权体现在哪？
- setExact/setWindow/setAndAllowWhileIdle 三者政策含义与功耗账。
- 五桶各自砍掉什么？判分与提桶的源码路径？
- 唤醒度四态与 PowerGroup/WakeLock/SuspendBlocker 三层关系。

附录 D 动手实验室（一天量）

实验一"Doze 现场"：`dumpsys deviceidle` 记录状态；灭屏静置（或 `dumpsys deviceidle force-idle deep`）观察任务/网络停摆；触发维护窗看放行——对照第二章图逐态打卡。

实验二"Job 全链"：写最小 JobDemo（约束：充电+不限网），`dumpsys jobscheduler | grep -A5 yours` 读约束位图与配额；拔充电线看"取消原因"。

实验三"桶威力"：`am set-standby-bucket demo restricted` 后发 Job/闹钟/后台网，逐项记录行为差异；再点开 App 看一步回 ACTIVE。

实验四"wakelock 狩猎"： Overnight 静置后 `dumpsys power | grep -A3 WAKE` 与 bugreport 的 kernel wakeup_sources 对照，找出真凶并归因（本册六章 + 书4 冻结联动）。

实验五"热复现"：`cmd thermalservice override-status 3` 后观察 AOD/亮度/帧率联动（书8 §7.1 的负输入现场）。

第九章 QuotaController 深读：配额窗口、桶预算与触发限流

> **设计洞察 · 学习目标**
> 1. 背出各桶配额表：10 分钟时长×不同窗口=速率杠杆；2. 走读 isWithinQuotaLocked 的三重豁免与三维判账；3. 理解 Timer/会话计费模型与 cutoff 闹钟；4. 说出充电豁免、1 分钟 20 个触发限流、InQuotaAlarmQueue 回款闹钟三条联动线

链路位置：第三章的 §3.2 给了 QuotaController 的"户口"（执行层计费员），§3.3 预告了 isWithinQuotaLocked 的三重结构；本章把它当成一台解剖台上的机器——从预算表（QcConstants）到判账（isWithinQuotaLocked）到记账（Timer/会话）再到催收（AlarmQueue），全程真实锚点

走读。它与第五章 App Standby 是"立法与执法"的关系：Standby 定桶（判分），Quota 按桶收税（执行预算）。

[缺图: bucket]

9.1 一张预算表：QcConstants 全家福

先看这张最容易背错的表——源码文件头注释（QuotaController.java:72-100）自己写明了设计："each app in each bucket is given 10 minutes to run within its respective time window"。**五个桶的时长额度一模一样，全靠窗口长度拉开销率。**

java JS/job/controllers/QuotaController.java:3110 起（QcConstants 骨架+讲解注释）

```
private static final long DEFAULT_ALLOWED_TIME_PER_PERIOD_EXEMPTED_MS =
10 * 60 * 1000L; // 10 minutes
private static final long DEFAULT_ALLOWED_TIME_PER_PERIOD_ACTIVE_MS =
10 * 60 * 1000L; // 10 minutes
private static final long DEFAULT_ALLOWED_TIME_PER_PERIOD_WORKING_MS =
10 * 60 * 1000L; // 10 minutes
private static final long DEFAULT_ALLOWED_TIME_PER_PERIOD_FREQUENT_MS =
10 * 60 * 1000L; // 10 minutes
private static final long DEFAULT_ALLOWED_TIME_PER_PERIOD_RARE_MS =
10 * 60 * 1000L; // 10 minutes
private static final long DEFAULT_ALLOWED_TIME_PER_PERIOD_RESTRICTED_MS =
10 * 60 * 1000L; // 10 minutes
// 讲解注释①：反直觉第一击——EXEMPTED 到 RESTRICTED 六档的时长额度全是
// 10 分钟！桶与桶的差别不在这，而在下面的窗口大小：同样的 10 分钟摊进
// 不同长度的滚动窗口，折算成"可持续速率"才拉开数量级。
private static final long DEFAULT_WINDOW_SIZE_WORKING_MS =
2 * 60 * 60 * 1000L; // 2 hours
private static final long DEFAULT_WINDOW_SIZE_FREQUENT_MS =
8 * 60 * 60 * 1000L; // 8 hours
private static final long DEFAULT_WINDOW_SIZE_RARE_MS =
24 * 60 * 60 * 1000L; // 24 hours
private static final long DEFAULT_WINDOW_SIZE_RESTRICTED_MS =
24 * 60 * 60 * 1000L; // 24 hours
// 讲解注释②：ACTIVE 的窗口=额度本身（10min 窗/10min 额→任意时刻都"有余粮"），
// WORKING=10min/2h（约 8%）、FREQUENT=10min/8h（约 2%）、
// RARE=10min/24h（约 0.7%）——滚动窗口意味着旧用量不断滚出窗口、
// 额度不断回笼，"一觉醒来配额又有了"的体感即由此而来。
private static final long DEFAULT_MAX_EXECUTION_TIME_MS = 4 * HOUR_IN_MILLIS;
// 讲解注释③：单段执行的硬顶 4 小时：无论桶多宽松，Timer.scheduleCutoff
// 会按它设闹钟强制断账——防止"低速率换长寿命"的擦边玩法。
private static final long DEFAULT_RATE_LIMITING_WINDOW_MS = MINUTE_IN_MILLIS;
private static final int DEFAULT_MAX_JOB_COUNT_PER_RATE_LIMITING_WINDOW = 20;
// 讲解注释④：触发限流（本册术语表）：不分桶，过去 1 分钟内最多启动
// 20 个 Job——专治"循环提交"炸弹；这是唯一对所有桶一视同仁的钳制。
private static final int DEFAULT_MAX_JOB_COUNT_WORKING =
(int) (60.0 * DEFAULT_WINDOW_SIZE_WORKING_MS / HOUR_IN_MILLIS); // =120/2h
private static final int DEFAULT_MAX_JOB_COUNT_FREQUENT =
(int) (25.0 * DEFAULT_WINDOW_SIZE_FREQUENT_MS / HOUR_IN_MILLIS); // =200/8h
private static final int DEFAULT_MAX_JOB_COUNT_RARE =
(int) (2.0 * DEFAULT_WINDOW_SIZE_RARE_MS / HOUR_IN_MILLIS); // =48/24h
private static final int DEFAULT_MAX_JOB_COUNT_RESTRICTED = 10;
```

```
private static final int DEFAULT_MAX_SESSION_COUNT_WORKING = 10; // 5/hr
private static final int DEFAULT_MAX_SESSION_COUNT_FREQUENT = 8; // 1/hr
private static final int DEFAULT_MAX_SESSION_COUNT_RARE = 3; // .125/hr
private static final int DEFAULT_MAX_SESSION_COUNT_RESTRICTED = 1; // 1/day
// 讲解注释⑤：次数双维：Job 数与 session 数（会话=一段连续后台执行，
// 间隔小于 DEFAULT_TIMING_SESSION_COALESCING_DURATION_MS=5 秒的
// 相邻执行并成一段）。RESTRICTED 一天 1 段 10 个——"每日窗口级"
// 惩罚（第五章）在这行常量里兑现，而不是在某个 if 里。
private static final long DEFAULT_IN_QUOTA_BUFFER_MS = 30 * 1000L;
// 讲解注释⑥：回款提前量：距额度恢复还差 30 秒就算"在额度内"——
// 避免"刚恢复就满载"的抖动，也让回款闹钟（9.4）有提前量可排。
// 加急账本（EJ）：24h 窗口内 EXEMPTED 45min→ACTIVE/WORKING 30min→
// FREQUENT/RARE 10min→RESTRICTED 5min（DEFAULT_EJ_LIMIT_*:3170-3175），
// 另有 quota bump（8h 窗最多 8 次，每次 +1min/+2 Job/+1 session）。
```

把注释①-⑤换算成一张速查表（数值均出自 QcConstants，可被 DeviceConfig 覆盖）：

桶	窗口	时长额度	折算速率	Job 数/会话数上限
ACTIVE	10 分钟	10 分钟	任意	75 个/75 段
WORKING_SET	2 小时	10 分钟	约 8%	120 个/10 段
FREQUENT	8 小时	10 分钟	约 2%	200 个/8 段
RARE	24 小时	10 分钟	约 0.7%	48 个/3 段
RESTRICTED	24 小时	10 分钟	约 0.7%	10 个/1 段

> **设计洞察 · 设计洞察："时长相同、窗口不同"的杠杆**
> 直觉设计是"越深的桶时长越短"，源码却选了"时长恒定、窗口递增"。妙处在滚动窗口的边际行为：时长差异化会让深桶出现"攒爆发的最优解"（憋着不跑，等一个长额度）；窗口差异化则让每个桶的速率上限连续、可预期，且桶升级（回 ACTIVE）的收益立即兑现——预算表是政策工具，Google 选了不易被博弈的那把。

9.2 isWithinQuotaLocked 三连走读

判账入口在 :887（第三章 §3.3 骨架的全文版）。注意它是两层三个重载：外层按 JobStatus 查豁免，内层按 (userId, packageName, bucket) 查账本。

java JS/job/controllers/QuotaController.java:887-955（节选+讲解注释）

```
boolean isWithinQuotaLocked(@NonNull final JobStatus jobStatus) {
if (!mIsEnabled) {
return true;
}
// 讲解注释①：Tare 启用时本控制器整体让位（mIsEnabled=false，见
// onConstantsUpdatedLocked:2927 的 USE_TARE_POLICY 分支）——
// 13 把"执行计费"升级成了独立子系统 Tare，Quota 是它的前身兼兜底。
final int standbyBucket = jobStatus.getEffectiveStandbyBucket();
// 讲解注释②：三重豁免按"或"短路：a）TOP 态启动的 Job（启动即终身
// 免费，之后退后台也不补收——"出发时的身份决定票价"）；b）源 uid
// 当前在前台（isUidInForeground:781，core uid 如 system 直接算前台）；
```

```
// c) 都不占才按桶查账（③）。
return isTopStartedJobLocked(jobStatus)
|| isUidInForeground(jobStatus.getSourceUid())
|| isWithinQuotaLocked(
jobStatus.getSourceUserId(), jobStatus.getSourcePackageName(), standbyBucket);
}
private boolean isQuotaFreeLocked(final int standbyBucket) {
// 讲解注释③：充电即免配额——插电等于不计功耗成本。唯一例外是
// RESTRICTED：它是行为惩罚不是资源管理，插电不解封（第五章）。
if (mService.isBatteryCharging()) {
return standbyBucket != RESTRICTED_INDEX;
}
return false;
}
boolean isWithinQuotaLocked(final int userId, @NonNull final String packageName,
final int standbyBucket) {
if (!mIsEnabled) {
return true;
}
if (standbyBucket == NEVER_INDEX) return false;
if (isQuotaFreeLocked(standbyBucket)) return true;
// 讲解注释④：真正的判账是三维"与"：时长（滚动窗口内剩余执行时间>0）、
// 次数（窗口内启动数<桶上限且不撞触发限流）、会话（窗口内段数<上限）
// ——三维全过才算有余量，缺一即"排队等下一班"（第三章 §3.2 的
// "排队不拒绝"哲学在判账层的落点）。
ExecutionStats stats = getExecutionStatsLocked(userId, packageName, standbyBucket);
return getRemainingExecutionTimeLocked(stats) > 0
&& isUnderJobCountQuotaLocked(stats, standbyBucket)
&& isUnderSessionCountQuotaLocked(stats, standbyBucket);
}
private boolean isUnderJobCountQuotaLocked(@NonNull ExecutionStats stats,
final int standbyBucket) {
final long now = sElapsedRealtimeClock.millis();
// 讲解注释⑤：触发限流的判法值得细品：不是硬比"窗口内计数<20"，
// 而是"限流窗已过期（jobRateLimitExpirationTimeElapsed<=now）或
// 窗内计数<20"——过期即视同清零，免去逐个滑出旧事件的扫描成本；
// ExecutionStats 是缓存，由 Timer 起停时作废重算（9.3）。
final boolean isUnderAllowedTimeQuota =
(stats.jobRateLimitExpirationTimeElapsed <= now
|| stats.jobCountInRateLimitingWindow < mMaxJobCountPerRateLimitingWindow);
return isUnderAllowedTimeQuota
&& stats.bgJobCountInWindow < stats.jobCountLimit;
}
```

设计洞察 · 逐段精读：为什么豁免判在查账之前

三重豁免全是"身份"判据（TOP 启动/前台/充电），判据零成本且语义稳定；查账是"账目"判据，要算缓存里的窗口统计。顺序即性能：绝大多数前台 App 的 Job 在第二行就短路返回，账本（mTimingEvents 时间线回放）只为后台常客服务——这与第六章 userActivityNoUpdateLocked "合法性三查在前、记账在后"是同一写法：先便宜后贵，先静态后动态。

9.3 Timer 与会话：执行时长是怎么入账的

判账读缓存，记账写缓存——写的人是内部类 Timer（:2101）。每个 (userId, packageName) 一把秒表，加急另有 mEJPkgTimers 一套。

java JS/job/controllers/QuotaController.java:2119/2286/2191（Timer 节选+讲解注释）

```
void startTrackingJobLocked(@NonNull JobStatus jobStatus) {
if (isTopStartedJobLocked(jobStatus)) {
// 讲解注释①：TOP 启动的 Job 不上秒表——9.2 注释②的"终身免费"
// 正是靠这里不记账兑现的：没进过账本，永远不会被收费。
return;
}
// Always maintain list of running jobs, even when quota is free.
if (mRunningBgJobs.add(jobStatus) && shouldTrackLocked()) {
mBgJobCount++;
if (mRegularJobTimer) {
incrementJobCountLocked(mPkg.userId, mPkg.packageName, 1);
}
if (mRunningBgJobs.size() == 1) {
// 讲解注释②：第一个后台 Job 启动=会话开场：记起始时间、清
// 借记修正量，并作废该包全部缓存的 ExecutionStats——因为
// "正在执行"这个事实本身改变了账面（窗口内时长在增长）。
mStartTimeElapsed = sElapsedRealtimeClock.millis();
mDebitAdjustment = 0;
if (mRegularJobTimer) {
invalidateAllExecutionStatsLocked(mPkg.userId, mPkg.packageName);
}
scheduleCutoff();
}
}
}
private void scheduleCutoff() {
synchronized (mLock) {
if (!isActive()) {
return;
}
// 讲解注释③：断账闹钟：getTimeUntilQuotaConsumedLocked(:1044)
// 沿 mTimingEvents 时间线回放窗口内每段会话，推出"再跑 X ms
// 就烧完额度"的时刻，用 handler 延迟消息排进未来。用 uptime
// 时钟是安全的——源码注释：Job 在跑则系统必然醒着，uptime 不走。
Message msg = mHandler.obtainMessage(
mRegularJobTimer ? MSG_REACHED_QUOTA : MSG_REACHED_EJ_QUOTA, mPkg);
final long timeRemainingMs = mRegularJobTimer
? getTimeUntilQuotaConsumedLocked(mPkg.userId, mPkg.packageName)
: getTimeUntilEJQuotaConsumedLocked(mPkg.userId, mPkg.packageName);
mHandler.sendMessageDelayed(msg, timeRemainingMs);
}
}
private void emitSessionLocked(long nowElapsed) {
if (mBgJobCount <= 0) {
return;
}
// 讲解注释④：会话出账：一段 TimingSession（起点/终点/Job 数）落进
// mTimingEvents 时间线，并累计 session 计数——判账的三个维度
```

```
// （时长/次数/段数）全部由这一个个 session 喂出来。
TimingSession ts = new TimingSession(mStartTimeElapsed, nowElapsed, mBgJobCount);
saveTimingSession(mPkg.userId, mPkg.packageName, ts, !mRegularJobTimer,
mDebitAdjustment);
mBgJobCount = 0;
cancelCutoff();
if (mRegularJobTimer) {
incrementTimingSessionCountLocked(mPkg.userId, mPkg.packageName);
}
}
```

出账时机也有讲究：stopTrackingJob（:2155）里只有"最后一个后台 Job 结束**且**当前不在充电"才 emitSessionLocked——充电期间执行不计 session（账照记、费不收），与 isQuotaFreeLocked 的豁免口径一致。两个相邻会话间隔小于 5 秒会被并算（coalescing），所以"循环 sleep(4s) 再跑一个 Job"刷不出更多会话额度——次数钳制不给擦边留门。

> **设计洞察 · 设计洞察：debit adjustment 与"公平计时"**
>
> updateDebitAdjustment(:2185) 记录的 mDebitAdjustment 是"系统欠 App 的时间"：Job 执行中被系统原因打断（如进入 Doze 被挂起），这段被扣掉的执行时间通过借记修正退还——计费按"真实执行"而非"名义持锁"。这与第三章"执行计费而非排队计费"互补：**排队的时段不算钱，跑一半被冻结的时段退款**，双向堵死不公平。

9.4 状态变化的三条联动线

预算表是静态的，联动线是动态的。三条线都收束到 maybeUpdateConstraint/重估约束，再交 JSS 重排队：

- **充电线**：电池状态变化 → onBatteryStateChangedLocked(:727) → handleNewChargingStateLocked(:1639)：TimerChargingUpdateFunctor(:1616) 遍历所有 Timer 决定"冻结计费还是恢复出账"，随后后台线程 maybeUpdateAllConstraintsLocked 全量重估——插拔线是配额世界的一次"全市场重定价"。
- **桶线**：StandbyTracker(:2473) 实现 AppIdleStateChangeListener，桶变更即重估该包全部 Job 并 rescheduleCutoff——源码头注释明说 "When a job's bucket changes, its new quota is immediately applied to it"。第五章的判分到这里兑现成新预算。
- **回款线**：InQuotaAlarmQueue(:2891，ALARM_TAG_QUOTA_CHECK=*job.quota_check*) 按 Package 预约"预计回到额度内的时刻"，到点发 MSG_CHECK_PACKAGE 重估——超配额的 Job 不占任何轮询，全靠这口闹钟唤醒；MIN_QUOTA_CHECK_DELAY 默认 1 分钟是它的最小节流。

text 一条 Job 的配额一生（读码归纳）

```
schedule → maybeStartTrackingJobLocked(:598)：入账+首判（:613 isWithinQuota）
在额度内 → setConstraintSatisfied(true) → 就绪池可执行（第三章管线）
超额度 → setConstraintSatisfied(false) + InQuotaAlarmQueue 预约回款闹钟
开始执行 → Timer.startTrackingJobLocked：第一个 Job 开会话、作废缓存、
scheduleCutoff 断账闹钟；每次执行 incrementJobCount（撞触发限流即停）
执行结束 → stopTrackingJob：最后一个结束且非充电 → emitSessionLocked 出账
桶变更/充电变更 → 重估+rescheduleCutoff（预算表换挡）
回款闹钟到点 → MSG_CHECK_PACKAGE → 重估 → 回到额度 → onControllerStateChanged
```

```
→ JSS 重排队 → "下一班再走"（第三章 §3.3 注释③的闭环）
// 讲解注释：观察全流程的窗口是 dumpsys jobscheduler 的 QuotaController 段
//  （dump:4116 起）：TimingEvents 时间线、每包 ExecutionStats、Timer 现场全在。
```

9.5 交叉引用与实验

- 回看第三章图 job-pipeline：Quota 是十二闸门的最后一道"计费闸"；本章解释了闸门的价目表。
- 与第二章的分工：Doze 维护窗内 Job 被放宽（DeviceIdleJobsController），但**放宽不等于免费**——维护窗内的执行照样过 Quota 记账，两个维度正交。
- 实验：`am set-standby-bucket <pkg> restricted` 后让 App 跑 11 个短 Job，`dumpsys jobscheduler | grep -A 8 QuotaController` 应能看到第 11 个停在 jobCountLimit（DEFAULT_MAX_JOB_COUNT_RESTRICTED=10）。

第十章 WakeLock 的一生：从 acquire 到内核唤醒源

设计洞察 · 学习目标

1. 走读客户端双计数器（mInternalCount/mExternalCount）与 token 双重身份；2. 逐行理解 acquireWakeLockInternal 的去重/记账/资格审查/顺序不变式；3. 说清 WakeLock 摘要层与 SuspendBlocker 物理层的换算；4. 记住 PARTIAL 锁三种停发场景与 60 秒长锁哨兵

链路位置：第六章 §6.1 给了 WakeLock 汇总层（updateWakeLockSummaryLocked）的户口；本章补全它的一生——客户端 acquire() 到服务端入账，再到 SuspendBlocker 换算成内核唤醒源，最后是释放、死亡与剥夺。它是全书"翻译主线"的又一个标本：开发者的一句 `acquire()`，被翻译成 uid 记账、按位或摘要、native 拦截三层时序。

[缺图: wakelock]

10.1 客户端半场：双计数器与 token

java frameworks/base/core/java/android/os/PowerManager.java:2922-3022（节选+讲解注释）

```
public void acquire() {
synchronized (mToken) {
acquireLocked();
}
}
// 讲解注释①：客户端的锁对象是 mToken——new WakeLock 时自建的匿名 Binder，
// 两重身份：服务端去重的主键（同一 token 重复 acquire 只更新属性）+
// 死亡通知载体（进程崩溃即自动回收，10.3）。
public void acquire(long timeout) {
synchronized (mToken) {
acquireLocked();
mHandler.postDelayed(mReleaser, timeout);
}
}
// 讲解注释②：带超时的 acquire=先拿锁再排 mReleaser——超时释放是客户端
// 自纠机制，不是服务端兜底；服务端对"忘 release"只报告不挽救（10.3）。
```

```
private void acquireLocked() {
mInternalCount++;
mExternalCount++;
if (!mRefCounted || mInternalCount == 1) {
// 讲解注释③：默认引用计数开启：只有 0→1 才真正跨 binder；非计数
// 模式下每次 acquire 都重发——源码注释点名 keyguard 的锁曾被
// PMS 强制释放而自己不知道，所以"已 held 也重发"以强制再拿。
mHandler.removeCallbacks(mReleaser);
Trace.asyncTraceBegin(Trace.TRACE_TAG_POWER, mTraceName, 0);
try {
mService.acquireWakeLock(mToken, mFlags, mTag, mPackageName, mWorkSource,
mHistoryTag, mDisplayId, mCallback);
} catch (RemoteException e) {
throw e.rethrowFromSystemServer();
}
mHeld = true;
}
}
public void release(int flags) {
synchronized (mToken) {
if (mInternalCount > 0) {
mInternalCount--;
}
if ((flags & RELEASE_FLAG_TIMEOUT) == 0) {
mExternalCount--;
}
// 讲解注释④：双计数器分工：mInternalCount 管引用语义（减到 0 才真
// 释放）；mExternalCount 管开发者纪律——普通 release 扣它，
// mReleaser 的超时释放带 RELEASE_FLAG_TIMEOUT 不扣（超时不算
// 用户失手），多释放直接抛 "WakeLock under-locked"。
if (!mRefCounted || mInternalCount == 0) {
mHandler.removeCallbacks(mReleaser);
if (mHeld) {
Trace.asyncTraceEnd(Trace.TRACE_TAG_POWER, mTraceName, 0);
try {
mService.releaseWakeLock(mToken, flags);
} catch (RemoteException e) {
throw e.rethrowFromSystemServer();
}
mHeld = false;
}
}
if (mRefCounted && mExternalCount < 0) {
throw new RuntimeException("WakeLock under-locked " + mTag);
}
}
}
private final Runnable mReleaser = () -> release(RELEASE_FLAG_TIMEOUT);
protected void finalize() throws Throwable {
...
// 讲解注释⑤：最廉价的泄漏检测器：持锁对象被 GC 回收仍未释放 →
// wtf 日志 "WakeLock finalized while still held: <tag>"——
// bugreport 里搜这句就是泄漏名单，tag 即 acquire 时的字符串。
Log.wtf(TAG, "WakeLock finalized while still held: " + mTag);
}
```

> **设计洞察 · 设计洞察：mToken 为什么必须是 Binder 对象**
> 用普通对象做 key 也能去重，但 Binder 对象额外买到两样东西：内核级死亡通知（对方进程死亡，服务端 handleWakeLockDeath 自动回收，**泄漏随崩溃自愈**）与跨进程稳定的身份（IBinder 的 proxy 在服务端映射到同一 binder 节点）。代价是每个 WakeLock 多一次 binder 对象分配——用一致性换健壮性，是全书反复出现的 binder 计价学（书3 §六）。

10.2 服务端半场：acquireWakeLockInternal 逐行

java services/core/.../power/PowerManagerService.java:1472-1548（节选+讲解注释）

```
private void acquireWakeLockInternal(IBinder lock, int displayId, int flags, String tag,
String packageName, WorkSource ws, String historyTag, int uid, int pid,
@Nullable IWakeLockCallback callback) {
boolean isCallerPrivileged = false;
try {
ApplicationInfo appInfo = mContext.getPackageManager().getApplicationInfo(packageName,
PackageManager.ApplicationInfoFlags.of(0));
isCallerPrivileged = appInfo.uid == uid && appInfo.isPrivilegedApp();
} catch (PackageManager.NameNotFoundException e) {
// assume app is not privileged
}
// 讲解注释①：包名与调用 uid 必须自洽才认特权——冒名包名查到的
// uid 对不上自己，拿不到 ACQUIRE_CAUSES_WAKEUP 等特权待遇。
synchronized (mLock) {
...
WakeLock wakeLock;
int index = findWakeLockIndexLocked(lock);
boolean notifyAcquire;
if (index >= 0) {
// 讲解注释②：token 命中旧账→不是新增而是幂等修正：属性变了
// 就 notifyWakeLockChangingLocked+updateProperties——非计数
// 模式的重发（10.1 注释③）在这里收敛成一次属性更新。
wakeLock = mWakeLocks.get(index);
if (!wakeLock.hasSameProperties(flags, tag, ws, uid, pid, callback)) {
notifyWakeLockChangingLocked(wakeLock, flags, tag, packageName,
uid, pid, ws, historyTag, callback);
wakeLock.updateProperties(flags, tag, packageName, ws, historyTag, uid, pid,
callback);
}
notifyAcquire = false;
} else {
// 讲解注释③：新账三件事：每 uid 一份的 UidState 记人头
//  （mNumWakeLocks，release 时减、清零即移除账本）；new
// WakeLock（:5030，实现 IBinder.DeathRecipient）挂进
// mWakeLocks；立刻做剥夺资格审查（10.4）——Doze 中拿锁
// 可能当场就是"停发"状态。
UidState state = mUidState.get(uid);
if (state == null) {
state = new UidState(uid);
state.mProcState = ActivityManager.PROCESS_STATE_NONEXISTENT;
mUidState.put(uid, state);
}
```

```
state.mNumWakeLocks++;
wakeLock = new WakeLock(lock, displayId, flags, tag, packageName, ws, historyTag,
uid, pid, state, callback);
mWakeLocks.add(wakeLock);
setWakeLockDisabledStateLocked(wakeLock);
notifyAcquire = true;
}
// 讲解注释④：ACQUIRE_CAUSES_WAKEUP 在这里兑现：仅屏幕类锁+
// 特权应用或 TURN_SCREEN_ON appops 才允许"拿锁即亮屏"，且
// 13 里只唤醒该锁所属 PowerGroup（INVALID_GROUP 才全组亮）——
// 第六章 PowerGroup 的"多屏各管各"在唤醒路径同样生效。
applyWakeLockFlagsOnAcquireLocked(wakeLock, isCallerPrivileged);
mDirty |= DIRTY_WAKE_LOCKS;
updatePowerStateLocked();
// 讲解注释⑤：顺序不变式：先结算（updatePowerStateLocked 可能已
// 让 native 层挂上内核 wakelock），后 notifyWakeLockAcquiredLocked
// （BatteryStats 记账）——源码注释：先记账后挂内核有"系统在
// 中间睡过去"的竞态。记账永远排在物理生效之后。
if (notifyAcquire) {
notifyWakeLockAcquiredLocked(wakeLock);
}
}
}
```

服务端的收尾三兄弟：releaseWakeLockInternal(:1643) 按索引摘账、unlinkToDeath、置 disabled 后 removeWakeLockLocked(:1683)；handleWakeLockDeath(:1663) 是死亡回调的落点——客户端进程任何形式的退出（crash/被杀/正常退出）都会触发，**WakeLock 不会随进程死亡而泄漏**。但注意：回收的是"这把锁"，进程若在 onDestroy 前持锁做事（如 sync 适配器），被杀瞬间做的事照样中断——泄漏防的是锁，不是数据完整性。

> **设计洞察 · 逐段精读：UidState 为什么单独记人头**
>
> release 路径 removeWakeLockLocked(:1683) 里：mNumWakeLocks-- 到 0 且进程状态为 NONEXISTENT 才把 UidState 从账本移除。这个"人头账"服务两个消费者：PowerManagerService 按 uid 判断"是否还有锁"（如 setDeviceIdleMode 时批量重估），BatteryStats 按 uid 归因（dumpsys batterystats 的 Wake lock 段以 uid 聚合）。**锁是层的，账是 uid 的**——两层账本各干各的，dumpsys 里看到的聚合视图即由此拼出。

10.3 停发与哨兵：剥夺逻辑与 60 秒长锁报告

text PARTIAL 锁的三种"停发"与一个哨兵（读码归纳）

```
setWakeLockDisabledStateLocked(:4017) 只处理 PARTIAL_WAKE_LOCK，应用 uid
（FIRST_APPLICATION_UID 以上）才受钳制，三种剥夺按"或"叠加：
① 缓存进程：mConstants.NO_CACHED_WAKE_LOCKS 开启时，uid 非活跃
（mActive=false）且进程状态低于 PROCESS_STATE_RECEIVER → 停发
——缓存进程的锁直接失效，"后台挂着锁防杀"被釜底抽薪；
② Doze：mDeviceIdleMode 时，不在 mDeviceIdleWhitelist/临时白名单、
且进程状态低于 BOUND_FOREGROUND_SERVICE → 停发——第二章
"深睡中 PARTIAL 也挂起，白名单例外"的源码实体就是这两行；
③ LowPowerStandby：mLowPowerStandbyActive 且不在 allowlist、
进程状态低于 BOUND_TOP → 停发（第六章 §6.2 的新中间态）。
```

```
// 讲解注释①：剥夺≠释放：mDisabled=true 只是不再计入摘要（摘要算完后
// SuspendBlocker 拿不到 WAKE_LOCK_CPU 位），锁本身还在账本里；
// updateWakeLockDisabledStatesLocked(:3992) 在状态解除时按需补发
// notify——BatteryStats 视角是"释放又拿回"，账目连续。
// 讲解注释②：哨兵：MIN_LONG_WAKE_CHECK_INTERVAL=60 秒（:233）。
// checkForLongWakeLocks(:2688) 扫描持锁超 60 秒的 PARTIAL 锁，
// notifyWakeLockLongStartedLocked(:1820) 上报 BatteryStats 打上
// long wakelock 标记——dumpsys batterystats 里 "long wake lock"
// 由此而来；这是系统对"长时间持锁"唯一的主动报告通道。
```

10.4 与第六章的拼图

第六章 §6.1 的注释③说"WakeLock 汇总→SuspendBlocker"，本章补全了语义层到物理层的最后一步换算（详情在第十二章 §12.3 结算器走读）：`mWakeLockSummary & WAKE_LOCK_CPU` 决定 mWakeLockSuspendBlocker 是否持有——**只有 PARTIAL 家族（含 DOZE/DRAW 等变体的 CPU 位）能拦 CPU 挂起，屏幕类锁走显示侧的 needSuspendBlockerLocked**。实验（附录 D 实验四）里 `dumpsys power | grep -A 3 WAKE` 看到的 Wake Locks 列表就是 mWakeLocks 账本的投影，每行的 tag/uid/held 时间都能对回本章两张代码清单。

第十一章 BatteryService：电量数据的来路与六路下游

设计洞察 · 学习目标

1. 描述 health HAL→HealthServiceWrapper→BatteryService.update 的注册与回调链；2. 走读 update/processValuesLocked 的真源替换与判变；3. 列出一次电量变化的六路下游；4. 用 dumpsys battery unplug/reset 做调试实验

链路位置：前十章都在讲"电量数据的消费者"（Job 的充电约束、Doze 的充电退出、省电模式的触发、耗电统计的归因），本章讲**唯一的供给方**——BatteryService。全系统所有上层看到的电量，都是 vendor health HAL 上报的 HealthInfo 在 framework 侧的投影；理解这一条供给线，前面所有"充电中/电量低"的行为才有第一因。

11.1 来路：registerHealthCallback 与双轨 HAL

java services/core/.../BatteryService.java:263-297（节选+讲解注释）

```
private void registerHealthCallback() {
traceBegin("HealthInitWrapper");
// IHealth is lazily retrieved.
// 讲解注释①：IHealth 懒获取：注册的是"服务出现"通知而非立刻拿句柄——
// health HAL 进程（vendor）可能比 system_server 晚起，晚到不误事。
try {
mHealthServiceWrapper = HealthServiceWrapper.create(this::update);
} catch (RemoteException ex) {
Slog.e(TAG, "health: cannot register callback. (RemoteException)");
throw ex.rethrowFromSystemServer();
} catch (NoSuchElementException ex) {
// 讲解注释②：AIDL+HIDL 双轨都找不到 health 服务 → 直接抛异常：
```

```
// 没有电量与电池温度数据的系统宁可 crash 循环重启（安全关机、
// 热兜底都依赖这份数据，宁可不可用不可错）。
Slog.e(TAG, "health: cannot register callback. (no supported health HAL service)");
throw ex;
} finally {
traceEnd();
}
traceBegin("HealthInitWaitUpdate");
// init register for new service notifications, and IServiceManager should return the
// existing service in a near future. Wait for this.update() to instantiate
// the initial mHealthInfo.
long beforeWait = SystemClock.uptimeMillis();
synchronized (mLock) {
while (mHealthInfo == null) {
// 讲解注释③：首帧等待：onStart 之后、第一帧 HealthInfo 到达之前，
// 系统不能对外报告任何电量（BatteryManager 查询全走 mHealthInfo）
// ——循环 wait(HEALTH_HAL_WAIT_MS) 阻塞到 update() 的 notifyAll。
Slog.i(TAG, "health: Waited " + (SystemClock.uptimeMillis() - beforeWait) +
"ms for callbacks. Waiting another " + HEALTH_HAL_WAIT_MS + " ms...");
mLock.wait(HEALTH_HAL_WAIT_MS);
}
}
Slog.i(TAG, "health: Waited " + (SystemClock.uptimeMillis() - beforeWait)
+ "ms and received the update.");
}
```

create 的内幕在 health/HealthServiceWrapper.java:109-117：先 `new HealthServiceWrapperAidl(...)`，`catch (NoSuchElementException)` 后回退 `new HealthServiceWrapperHidl(...)`——**AIDL 优先、HIDL 兜底**的双轨是 Mainline 时代 HAL 接口迁移的标准姿势（与本册前言的"JobScheduler 进 apex 模块"同一时代背景：接口层快速演进、实现层向后兼容）。

11.2 结算：update 与 processValuesLocked

java services/core/.../BatteryService.java:423-506（节选+讲解注释）

```
private void update(android.hardware.health.HealthInfo info) {
traceBegin("HealthInfoUpdate");
// 讲解注释①：四个 trace counter 先行打点：电量计数/电流/插头类型/
// 状态——perfetto 上"电量轨"的四个通道就是这四行。
Trace.traceCounter(
Trace.TRACE_TAG_POWER, "BatteryChargeCounter", info.batteryChargeCounterUah);
Trace.traceCounter(Trace.TRACE_TAG_POWER, "BatteryCurrent", info.batteryCurrentMicroamps);
Trace.traceCounter(Trace.TRACE_TAG_POWER, "PlugType", plugType(info));
Trace.traceCounter(Trace.TRACE_TAG_POWER, "BatteryStatus", info.batteryStatus);
synchronized (mLock) {
if (!mUpdatesStopped) {
mHealthInfo = info;
// 讲解注释②：真源替换：从这行起全系统的新电量都是 info 的投影；
// mUpdatesStopped（dumpsys battery unplug 的调试态）时只把
// 新值拷进影子账本 mLastHealthInfo，正式账本冻结。
processValuesLocked(false);
mLock.notifyAll(); // for any waiters on new info
```

```
// 讲解注释③：notifyAll 唤醒 11.1 注释③的首帧等待者——注册链
// 与数据链在 mLock 上完成交接。
} else {
copyV1Battery(mLastHealthInfo, info);
}
}
traceEnd();
}
private void processValuesLocked(boolean force) {
mBatteryLevelCritical =
mHealthInfo.batteryStatus != BatteryManager.BATTERY_STATUS_UNKNOWN
&& mHealthInfo.batteryLevel <= mCriticalBatteryLevel;
mPlugType = plugType(mHealthInfo);
// 讲解注释④：下游第一站 BatteryStats：每次电量变化先喂 batterystats
// ——dumpsys batterystats 的电量曲线、耗电排行的时间轴基准全在这
// （排障手册场景五的"耗电归因"就靠这条时间轴对齐）。
try {
mBatteryStats.setBatteryState(
mHealthInfo.batteryStatus,
mHealthInfo.batteryHealth,
mPlugType,
mHealthInfo.batteryLevel,
mHealthInfo.batteryTemperatureTenthsCelsius,
mHealthInfo.batteryVoltageMillivolts,
mHealthInfo.batteryChargeCounterUah,
mHealthInfo.batteryFullChargeUah,
mHealthInfo.batteryChargeTimeToFullNowSeconds);
} catch (RemoteException e) {
// Should never happen.
}
// 讲解注释⑤：安全双闸同步判（不等广播不排队）：电量临界且未充电 →
// 发 ACTION_REQUEST_SHUTDOWN（SHUTDOWN_LOW_BATTERY）；电池温度超
// mShutdownBatteryTemperature（默认 68□）→ SHUTDOWN_BATTERY_THERMAL_
// STATE——第七章 ThermalManagerService 管"降速"，这里管"关机"，
// 两道热防线互为表里。
shutdownIfNoPowerLocked();
shutdownIfOverTempLocked();
// 讲解注释⑥：判变：force 或 11 个字段（状态/健康/在位/电量/插头/电压/
// 温度/充电电流/充电电压/计数器/无效充电器）任一变化才继续——电量
// 每秒都在微变，广播只发"有意义的变化"；插拔瞬间另记充电/放电
// 起始账（mChargeStartLevel/mDischargeStartTime）供指标打点。
if (force
|| (mHealthInfo.batteryStatus != mLastBatteryStatus
|| mHealthInfo.batteryLevel != mLastBatteryLevel
|| mPlugType != mLastPlugType
...)) {
...
sendBatteryChangedIntentLocked();
}
}
```

11.3 出口：粘性广播与六路下游

java services/core/.../BatteryService.java:682-712（节选+讲解注释）

```
private void sendBatteryChangedIntentLocked() {
final Intent intent = new Intent(Intent.ACTION_BATTERY_CHANGED);
intent.addFlags(Intent.FLAG_RECEIVER_REGISTERED_ONLY
| Intent.FLAG_RECEIVER_REPLACE_PENDING);
// 讲解注释①：两面旗即政策：REGISTERED_ONLY——静态接收者不叫醒
// （杀进程不会为"读电量"这种高频状态买单）；REPLACE_PENDING——
// 上一份没发完就作废，永远只保留最新一份（电量事件风暴下不积压）。
int icon = getIconLocked(mHealthInfo.batteryLevel);
intent.putExtra(BatteryManager.EXTRA_SEQUENCE, mSequence);
intent.putExtra(BatteryManager.EXTRA_STATUS, mHealthInfo.batteryStatus);
intent.putExtra(BatteryManager.EXTRA_LEVEL, mHealthInfo.batteryLevel);
intent.putExtra(BatteryManager.EXTRA_PLUGGED, mPlugType);
intent.putExtra(BatteryManager.EXTRA_TEMPERATURE,
mHealthInfo.batteryTemperatureTenthsCelsius);
...
mHandler.post(() -> ActivityManager.broadcastStickyIntent(intent, UserHandle.USER_ALL));
}
```

一次电量变化（过了判变）的全景下游，收束成一张账（读码归纳）：

- **BatteryStats**：setBatteryState(:478)——耗电统计的时间轴（本章 11.2 注释④，排障手册的基础账本）；
- **安全双闸**：shutdownIfNoPowerLocked(:382)/shutdownIfOverTempLocked(:402)——ACTION_REQUEST_SHUTDOWN 走 startActivity（systemReady 后才安全）；
- **粘性广播**：ACTION_BATTERY_CHANGED(:682)——全 App 的电量快照（registerReceiver(null, filter) 即取当前值）；
- **JobScheduler**：BatteryController 收 ACTION_POWER_CONNECTED/DISCONNECTED（BatteryController.java:220）并查 BatteryManagerInternal（BatteryService 以 LocalService 发布，:236）置 charging 约束（第三章闸门），联动 QuotaController.isQuotaFreeLocked——充电即免配额（第九章 9.2 注释③）；
- **Doze**：DeviceIdleController 收广播退出深睡（第二章 ACTIVE 转移：充电是三刺激之一）；
- **省电模式**：onBootPhase(:243) 注册的 ContentObserver 盯 LOW_POWER_MODE_TRIGGER_LEVEL（:287-302），低电量阈值变化→updateBatteryWarningLevelLocked→processValuesLocked(true)——第六章 §6.3 的 BatterySaverStateMachine 由此获得触发信号。

设计洞察 · 逐段精读：粘性广播是"状态快照总线"而非事件流

ACTION_BATTERY_CHANGED 的三件套（REGISTERED_ONLY/REPLACE_PENDING/sticky）合起来表达一种语义：接收者拿到的是"此刻快照"，不是"变化历史"。想读电量根本不用等广播——registerReceiver 传 null receiver 立即回粘性值。这也是为什么很多"电量监听"写法是错的：在 onReceive 里再读 EXTRA_LEVEL 永远是最新值而不是触发值——快照总线没有"那一帧"。与书2 广播系统的 sticky 退役政策对照：BatteryService 是少数保留者，理由正是"全系统状态快照"的公共品属性。

11.4 调试后门与实验

BatteryService 的 shell 命令（dump:860 起）是全 AOSP 最友好的调试面之一：`dumpsys battery unplug [-f]` 冻结电池态（mUpdatesStopped=true，update() 只进影子账，全系统以为拔了线——**验证"拔线即取消 Job"（第三章）与"充电退出 Doze"（第二章）的最快方法**）；`dumpsys battery reset` 解冻恢复；`dumpsys battery` 查看当前 mHealthInfo 全部字段。实验：插着线执行 unplug 后立刻 `dumpsys deviceidle`，观察白名单外的 Job 进入"受 Doze 影响"的候场状态——一条命令制造出本册一半章节的实验条件。

第十二章 灭屏全链：从一次超时到内核挂起

> **设计洞察 · 学习目标**
> 1. 背出 updatePowerStateLocked 六相结算器的顺序与理由；2. 走读"该睡觉了吗"三分支（inattentive/dream/doze）与 PowerGroup.sleepLocked；3. 描述 Notifier 的早/晚两段交互切换；4. 走读 updateSuspendBlockerLocked 的先取后放不变式

链路位置：第六章 §6.1 走读了 userActivity 的"记账"，第十章走读了 WakeLock 的"入账"，本章走读最后一块：**结算**。触摸停了、倒计时到了、最后一把锁放了——这些事件如何汇成一次真实的灭屏与 CPU 挂起，是 PMS 全书的总装线。

12.1 六相结算器：updatePowerStateLocked

PMS 的主循环只有一句话：**任何事件都只改 mDirty 位，统一由 updatePowerStateLocked(:2330) 全量重算**——第六章"先记账后结算"模式的总开关。源码注释自己写明："We centralize them here so that we can recompute the power state completely each time something important changes"。

java services/core/.../power/PowerManagerService.java:2330-2396（节选+讲解注释）

```
private void updatePowerStateLocked() {
if (!mSystemReady || mDirty == 0 || mUpdatePowerStateInProgress) {
return;
}
// 讲解注释①：三重守门：系统未就绪不结算、无事（mDirty==0）不结算、
// 已在结算中直接返回（防重入——结算过程本身会再置 mDirty）。
Trace.traceBegin(Trace.TRACE_TAG_POWER, "updatePowerState");
mUpdatePowerStateInProgress = true;
try {
// Phase 0: Basic state updates.
updateIsPoweredLocked(mDirty);
updateStayOnLocked(mDirty);
updateScreenBrightnessBoostLocked(mDirty);
// 讲解注释②：Phase 0 基础项：插电状态/开发选项"充电不锁屏"/
// 亮度提升——它们影响后面的"该不该睡"判据，故最先算。
// Phase 1: Update wakefulness.
// Loop because the wake lock and user activity computations are influenced
```

```
// by changes in wakefulness.
final long now = mClock.uptimeMillis();
int dirtyPhase2 = 0;
for (;;) {
int dirtyPhase1 = mDirty;
dirtyPhase2 |= dirtyPhase1;
mDirty = 0;
updateWakeLockSummaryLocked(dirtyPhase1);
updateUserActivitySummaryLocked(now, dirtyPhase1);
updateAttentiveStateLocked(now, dirtyPhase1);
if (!updateWakefulnessLocked(dirtyPhase1)) {
break;
}
}
// 讲解注释③：Phase 1 是唯一带循环的相位：唤醒度一变，"哪些锁
// 还有意义、用户活动倒计时还剩多少"都会跟着变——所以反复
// 重算直到不再转移为止（不动点）。第十章 10.2 的摘要层在
// 这里的每轮循环中被重刷。
// Phase 2: Lock profiles that became inactive/not kept awake.
updateProfilesLocked(now);
// Phase 3: Update power state of all PowerGroups.
final boolean powerGroupsBecameReady = updatePowerGroupsLocked(dirtyPhase2);
// 讲解注释④：Phase 2/3：工作资料（profile）先锁，然后逐个
// PowerGroup 落实显示策略（第六章：多屏各自醒睡）。
// Phase 4: Update dream state (depends on power group ready signal).
updateDreamLocked(dirtyPhase2, powerGroupsBecameReady);
// Phase 5: Send notifications, if needed.
finishWakefulnessChangeIfNeededLocked();
// 讲解注释⑤：Phase 4/5：屏保依赖"显示组就绪"信号；广播与回调
//  （Notifier）在所有账本落定后才发——外部世界看到的顺序与
// 内部结算顺序一致，不存在"广播已到账本未平"。
// Phase 6: Update suspend blocker.
// Because we might release the last suspend blocker here, we need to make sure
// we finished everything else first!
updateSuspendBlockerLocked();
// 讲解注释⑥：Phase 6 必须最后：这里可能放掉最后一个 SuspendBlocker
// ——源码注释原话 "we finished everything else first!"。物理
// 挂起是结算的句号，句号前所有记账必须完成。
} finally {
Trace.traceEnd(Trace.TRACE_TAG_POWER);
mUpdatePowerStateInProgress = false;
}
}
```

12.2 该睡觉了吗：BedTime 三分支

超时事件的起点是 handler 消息 MSG_USER_ACTIVITY_TIMEOUT(:161)：userActivity 刷新的倒计时到点 → handleUserActivityTimeout(:2965) 只做一件事 `mDirty |= DIRTY_USER_ACTIVITY` 后进结算器——又是记账模式。Phase 1 的循环里，updateWakefulnessLocked(:3054) 对每个醒着的 PowerGroup 问"到睡觉时间了吗"：

java services/core/.../power/PowerManagerService.java:3054-3092（节选+讲解注释）

```
private boolean updateWakefulnessLocked(int dirty) {
```

```
...
for (int idx = 0; idx < mPowerGroups.size(); idx++) {
final PowerGroup powerGroup = mPowerGroups.valueAt(idx);
if (!(powerGroup.getWakefulnessLocked() == WAKEFULNESS_AWAKE
&& isItBedTimeYetLocked(powerGroup))) {
continue;
}
// 讲解注释①：候选资格：该组醒着且"到点"（isItBedTimeYetLocked:
// 3106=没有被 keep awake 的锁/活动撑着）。第十章的锁摘要
//  （STAY_AWAKE 位）就在这一步反过来阻止入睡。
if (isAttentiveTimeoutExpired(powerGroup, time)) {
// 讲解注释②：分支一：attentive 超时（"长时间无人"极简待机）
// → 直接睡（GO_TO_SLEEP_REASON_INATTENTIVE）——跳过屏保
// 与 doze，第六章 §6.2 LowPowerStandby 的入口之一。
changed = sleepPowerGroupLocked(powerGroup, time,
PowerManager.GO_TO_SLEEP_REASON_INATTENTIVE, Process.SYSTEM_UID);
} else if (shouldNapAtBedTimeLocked()) {
// 讲解注释③：分支二：设置开启"屏保"→ 做梦（DREAMING）——
// 醒着但降级，交互性保持。
changed = dreamPowerGroupLocked(powerGroup, time, Process.SYSTEM_UID);
} else {
// 讲解注释④：分支三（默认灭屏）：进入 DOZING——AOD 亮或灭的
// "睡前面"（第六章：AOD=DOZING 唤醒度）；从 DOZING 到
// ASLEEP 还有 sandman 阶段（显示 ready 后再沉底）。
changed = dozePowerGroupLocked(powerGroup, time,
PowerManager.GO_TO_SLEEP_REASON_TIMEOUT, Process.SYSTEM_UID);
}
}
return changed;
}
```

三个分支殊途同归于 PowerGroup 的 setWakefulnessLocked——以 sleepLocked(PowerGroup.java:269) 为例：合法性检查（时间早于上次醒来/已经睡着则拒绝）→ setSandmanSummonedLocked(睡神点名) → setWakefulnessLocked(WAKEFULNESS_ASLEEP)。**唤醒度是 PowerGroup 的属性**：一次"整机灭屏"实际是每个显示组各自的入睡，全局唤醒度由 updateGlobalWakefulnessLocked(:2082) 按 recalculate 汇总（折叠屏合盖那半屏先睡时，全局值跟随多数/主屏）。

12.3 广播与物理层：Notifier 两段式与 SuspendBlocker 先取后放

java services/core/.../power/Notifier.java:415-462 + PowerManagerService.java:3565-3620（节选+讲解注释）

```
public void onWakefulnessChangeStarted(final int wakefulness, int reason, long eventTime) {
final boolean interactive = PowerManagerInternal.isInteractive(wakefulness);
// 讲解注释①：第一段（早段）：交互性一变立刻动输入——setInteractive
// 让输入管线先于屏幕切换准备（唤醒路径输入要尽早、灭屏路径
// 输入要尽早停），并 noteInteractive 给 BatteryStats 记时间轴。
mHandler.post(new Runnable() {
@Override
public void run() {
mActivityManagerInternal.onWakefulnessChanged(wakefulness);
}
});
```

```
if (mInteractive != interactive) {
if (mInteractiveChanging) {
handleLateInteractiveChange();
}
mInputManagerInternal.setInteractive(interactive);
mInputMethodManagerInternal.setInteractive(interactive);
try {
mBatteryStats.noteInteractive(interactive);
} catch (RemoteException ex) { }
...
handleEarlyInteractiveChange();
}
}
// 讲解注释②：第二段（晚段）在 onWakefulnessChangeFinished(:463)：
// 显示真正 ready 后才做慢动作（亮屏广播 ACTION_SCREEN_ON、灭屏的
// goToSleep 广播）——"快动作先行、慢通知殿后"把用户体感延迟压到
// 最小；dumpsys power 里 last sleep reason 与广播时间戳的差值即两段间隙。
private void updateSuspendBlockerLocked() { // PMS:3565（Phase 6 总装）
final boolean needWakeLockSuspendBlocker = ((mWakeLockSummary & WAKE_LOCK_CPU) != 0);
final boolean needDisplaySuspendBlocker = needSuspendBlockerLocked();
final boolean autoSuspend = !needDisplaySuspendBlocker;
// 讲解注释③：换算点：第十章的锁摘要里只有 WAKE_LOCK_CPU 位（PARTIAL
// 家族汇总）直接拦 CPU 挂起；显示侧锁由 needSuspendBlockerLocked
// 另判；autoSuspend 交给 HAL 决定内核自动挂起的开关。
boolean interactive = false;
for (int idx = 0; idx < mPowerGroups.size() && !interactive; idx++) {
interactive = mPowerGroups.valueAt(idx).isBrightOrDimLocked();
}
// 先取后放：先挂需要的 blocker 再放多余的——两步之间不存在
// "无人拦截"的空窗，挂起资格从没有"闪断"的瞬间。
if (needWakeLockSuspendBlocker && !mHoldingWakeLockSuspendBlocker) {
mWakeLockSuspendBlocker.acquire();
mHoldingWakeLockSuspendBlocker = true;
}
...
if (mDecoupleHalInteractiveModeFromDisplayConfig) {
if (interactive || areAllPowerGroupsReadyLocked()) {
setHalInteractiveModeLocked(interactive);
}
}
// 讲解注释④：PARTIAL 锁只保 CPU 不保交互：interactive 由显示组
// bright/dim 决定——CPU 醒着、HAL non-interactive 的组合（后台
// Job 在跑但屏已灭）正是省电调度的理想态，vendor 可据此降频。
if (!needWakeLockSuspendBlocker && mHoldingWakeLockSuspendBlocker) {
mWakeLockSuspendBlocker.release();
mHoldingWakeLockSuspendBlocker = false;
}
// 讲解注释⑤：最后一把 PARTIAL 锁释放、且显示侧无需求 → 放
// SuspendBlocker → native 写 /sys/power/wake_unlock → 内核
// autosleep 获准挂起 CPU——第二章 Doze 深睡的物理前提。
}
```

> **设计洞察 · 设计洞察：六相顺序是一条"依赖拓扑"**
> 把结算器读成 DAG：基础状态→唤醒度（迭代到不动点）→profile→显示组→屏保→通知→物理挂起。每相只依赖前面相位的输出，通知晚于账本、物理晚于通知。**任何"为什么先 X 后 Y"的问题在 PMS 的答案几乎都是 Phase 编号**——面试中答"结算顺序即依赖顺序"并给出 Phase 6 注释原文，比背状态机图更能证明读过源码。

12.4 全链收拢与交叉引用

把四条链并成一句话收束全书：**健康 HAL 报来电量（第十一章）→ 电池状态改变让 Doze 退出/进入、让配额免费或收费（第九章/第二章）→ 灭屏超时把 PowerGroup 送进 DOZING/ASLEEP（本章）→ 未被白名单豁免的 PARTIAL 锁被停发（第十章）→ 最后一个 SuspendBlocker 释放，CPU 真正睡去**。后台经济学的"成本结算"至此闭环。验证实验：灭屏前 `dumpsys power` 记下 mWakefulness 与 Display Power state，等 30 秒后再 dump 一次，对照第十章 10.3 的停发逻辑与本章 Phase 1 循环，把每行输出归位到四条链之一。

教科书习题集（分级）

第一级：简答题（检验概念理解）

1. **本树的 JobScheduler/Alarm/Doze 源码在哪个路径？与 Mainline 模块是什么关系？**

参考答案要点：frameworks/base/apex/jobscheduler/（service/java/com/android/server 下含 job/alarm/controllers），它们是 com.android.scheduling 模块的实体（书1 §16.3 表的活证据）；旧教材的 services/core/.../job|alarm 路径在本树不存在（第一章）。

2. **QuotaController 的五桶配额表是什么？为什么"时长相同、窗口不同"反而是好设计？**

参考答案要点：六档时长额度全是 10 分钟/窗口，窗口 WORKING 2h、FREQUENT 8h、RARE/RESTRICTED 24h（ACTIVE 窗=额=任意时刻可跑），折算速率约 8%/2%/0.7%；次数双维 WORKING 120 个 10 段→RESTRICTED 10 个 1 段。窗口差异化不产生"憋爆发"的博弈最优解，且桶升级收益立即兑现（第九章 §9.1）。

3. **一个 Job 的配额"免费"有哪几种情形？各自在哪个方法里判？**

参考答案要点：TOP 态启动（isTopStartedJobLocked，终身免费）、源 uid 前台（isUidInForeground）、充电中除 RESTRICTED 外（isQuotaFreeLocked:903）、Tare 启用时整个控制器让位（mIsEnabled=false）——外层 isWithinQuotaLocked:887 按"或"短路（第九章 §9.2）。

4. **"触发限流"是什么？isUnderJobCountQuotaLocked 的判法为什么是"过期或小于 20"而不是硬比计数？**

参考答案要点：不分桶，过去 1 分钟内最多启动 20 个 Job（DEFAULT_MAX_JOB_COUNT_PER_RATE_LIMITING_WINDOW=20）专治循环提交；判法"

jobRateLimitExpirationTimeElapsed<=now 或窗内计数<20"让窗口过期即视同清零，免去逐个滑出旧事件的扫描（第九章 §9.2 注释⑤）。

5. **配额里的"会话"（session）如何定义？5 秒合并窗口堵住了什么擦边玩法？**

参考答案要点：一段连续后台执行（TimingSession：起点/终点/Job 数），相邻执行间隔小于 DEFAULT_TIMING_SESSION_COALESCING_DURATION_MS=5 秒并成一段；"循环 sleep(4s) 再跑一个 Job"刷不出更多会话额度——RESTRICTED 一天 1 段的钳制无法被碎片化绕开（第九章 §9.3）。

6. **PowerManager.WakeLock 的双计数器（mInternalCount/mExternalCount）如何分工？**

参考答案要点：mInternalCount 管引用语义（非 0→1 不跨 binder，减到 0 才真释放）；mExternalCount 管开发者纪律（普通 release 扣、mReleaser 的超时释放带 RELEASE_FLAG_TIMEOUT 不扣，多释放抛 under-locked）（第十章 §10.1 注释④）。

7. **客户端 WakeLock 的 mToken 为什么必须是 Binder 对象？服务端对同 token 重复 acquire 怎么处理？**

参考答案要点：Binder 对象买到内核级死亡通知（handleWakeLockDeath:1663 随进程崩溃自动回收）与跨进程稳定身份；服务端 findWakeLockIndexLocked 命中旧账走 updateProperties 幂等修正而非新增（acquireWakeLockInternal:1503 分支）——非计数模式的重发在此收敛（第十章 §10.1/§10.2）。

8. **PARTIAL 锁有哪三种"停发"场景？"剥夺"与"释放"的区别是什么？**

参考答案要点：缓存进程非活跃（低于 RECEIVER）、Doze 且不在两级白名单（低于 BOUND_FOREGROUND_SERVICE）、LowPowerStandby 不在 allowlist（低于 BOUND_TOP）——setWakeLockDisabledStateLocked:4017。剥夺只停摘要计数（mDisabled），锁仍在账本，解除后按需补发 notify（第十章 §10.3）。

9. **60 秒长锁哨兵是什么？它在 dumpsys batterystats 里对应什么？**

参考答案要点：MIN_LONG_WAKE_CHECK_INTERVAL=60 秒（PMS:233），checkForLongWakeLocks:2688 扫描持锁超时的 PARTIAL 锁并 notifyWakeLockLongStartedLocked 上报 BatteryStats；dumpsys batterystats 的 long wake lock 标记即来自它——系统对长时间持锁唯一的主动报告通道（第十章 §10.3 注释②）。

10. **BatteryService 的 health HAL 绑定为什么是"双轨"？为什么找不到服务就宁可 crash？**

参考答案要点：HealthServiceWrapper.create:109 先试 AIDL（android.hardware.health）再 catch NoSuchElementException 回退 HIDL 2.0——接口迁移期的标准双轨；无电量/电池温度数据则安全关机双闸失效，system_server 宁可重启也不带错运行（第十一章 §11.1 注释②）。

第二级：分析题（检验机制理解）

11. **追踪调用链：一个处于 RESTRICTED 桶的 Job 从"超配额"到"回款后执行"的完整链路，标出每一步的方法锚点。**

参考答案要点：maybeStartTrackingJobLocked:598 首判 isWithinQuota:613=false→setConstraintSatisfied(false)+InQuotaAlarmQueue(:2891) 按 Package 预约回款闹钟（ALARM_TAG_QUOTA_CHECK）→旧用量滚出 24h 窗口或额度回笼（IN_QUOTA_BUFFER 提前 30s）→闹钟到点 MSG_CHECK_PACKAGE 重估 →onControllerStateChanged→JSS maybeQueueReadyJobsForExecutionLocked 重新入池 →bind 执行（第三章四站）。

12. **追踪调用链：灭屏超时到最后一个 SuspendBlocker 释放的完整链，指出哪一步用了不动点循环。**

参考答案要点：倒计时到点 MSG_USER_ACTIVITY_TIMEOUT(:161)→handleUserActivityTimeout:2965 只置 DIRTY_USER_ACTIVITY→updatePowerStateLocked:2330 六相：Phase 1 内 updateWakefulnessLocked:3054 判 isItBedTimeYetLocked:3106→dozePowerGroupLocked→PowerGroup.setWakefulnessLocked→Notifier.onWakefulnessChangeStarted:415（早段）→Phase 6 updateSuspendBlockerLocked:3565 放 mWakeLockSuspendBlocker→wake_unlock；不动点循环在 Phase 1（唤醒度变化反过来影响锁摘要与用户活动摘要）。

13. **失败推演：某 App 在 RESTRICTED 桶、不充电，每秒 schedule 一个执行 10 秒的 Job 并立即跑。各道钳制分别在第几个 Job 后收网？**

参考答案要点：第一道 1 分钟 20 个的触发限流约在第 6 个 Job 后挡住第 7 个（1 分钟窗满）；10 个 Job 计数额度约 100 秒后耗尽（若限流窗放得开）；第 1 段会话持续占用把 24h 会话额度（1 段）用掉；最终系统态：Job 全部 setConstraintSatisfied(false)+回款闹钟排到窗口滚动点——三层钳制"速率→总量→会话"逐级收网（第九章 §9.1/§9.4）。

14. **失败推演：health HAL 进程 crash 期间，上层对电量的感知是什么？重启后如何自愈？**

参考答案要点：HAL 死后 update() 不再被调，mHealthInfo 冻结在旧值 ——ACTION_BATTERY_CHANGED 粘性快照持续广播旧状态、BatteryStats 时间轴停更；wrapper 的服务注册通知（IServiceManager onRegistration）在 HAL 重启后重绑并 scheduleUpdate 补发一帧；期间 BatteryService 不崩（区别于"注册时找不到就抛异常"——只有初始化失败才 crash，运行中断靠重绑自愈，第十一章 §11.1/§11.2）。

15. **失败推演：一个前台服务 App 持 PARTIAL 锁且进程状态长期是 BOUND_FOREGROUND_SERVICE，Doze 深睡会发生什么？**

参考答案要点：setWakeLockDisabledStateLocked 的 Doze 分支条件"procState 高于 BOUND_FOREGROUND_SERVICE 才停发"——前台服务恰好豁免，锁继续拦挂起，CPU 不睡、Doze 的深睡收益（挂起+wakeup 来源收敛）打折；但网络断/Job 窗口化照常（这两限不走 wakelock 通道）——"Doze 三限制"并非一荣俱荣，wakelock 维度由进程状态独立裁决（第二章 §2.2/第十章 §10.3）。

16. **跨册串联：本章哪三处机制依赖 binder 的进程语义？分别用了 binder 的哪个特性？**

参考答案要点：①WakeLock 的 token 死亡通知=IBinder.DeathRecipient（linkToDeath，书3 §六）；②BatteryService 双面发布=publishBinderService（对外 shell/App）+publishLocalService BatteryManagerInternal（同进程 JobScheduler 直查，省一次 binder）；③sticky 广播=AMS 的 binder 调用路径（broadcastStickyIntent）+REPLACE_PENDING 的服务端去重（第十章/第十一章/书3）。

17. **对比分析：isQuotaFreeLocked 对充电全免、唯独 RESTRICTED 不免。请从"资源管理 vs 行为惩罚"推演这条例外，并给出一条反例边界。**

参考答案要点：充电豁免的逻辑是"插电=功耗成本可忽略"（资源维度），而 RESTRICTED 的语义是用户/系统对 App 行为的不信任（行为维度），不随资源状况解封；反例边界：Doze 深睡对 wakelock 的停发也保留白名单豁免但同样不让"行为不端者"用免费额度连跑——凡"惩罚性限制"都不应被资源条件反向抵消，否则惩罚失去威慑（第九章 §9.2 注释③/第五章）。

第三级：设计题（检验架构能力）

18. **为"自动化测试农场"设计一套配额豁免策略：要求测试机跑任意 Job 不被限流，且不污染量产代码路径。**

参考答案要点：优先用现成旋钮：DeviceConfig 覆盖 QcConstants（MAX_JOB_COUNT_PER_RATE_LIMITING_WINDOW 上限受 MIN 值钳制 :3386、WINDOW_SIZE_*、ALLOWED_TIME_*）按 build 指纹下发；测试包入临时白名单（temp-allowlist）+插电态（isQuotaFreeLocked 免配额）+`am set-standby-bucket active` 三件套；兜底可切 USE_TARE_POLICY 让 Tare 接管或调 mIsEnabled 开关——量产路径零改动，全部旋钮已有 config 化出口（第九章 §9.1/§9.4）。

19. **Quota 的账本是 per-(userId, packageName) 的。若同一包被多个 uid 共享（sharedUserId/WorkSource 归因），配额应记谁头上？请给出记账键设计与归因规则。**

参考答案要点：维持 (userId, packageName) 为预算键但引入 WorkSource 细分：Timer 记账时按 jobStatus 的 WorkSource/WorkChain 归因到真实消耗方（acquireWakeLockInternal 已示范 WorkChain 取 attributionTag 的手法），预算判定用包级、报表与追责用 uid 级——预算键改动会破坏桶语义（桶按包判分），归因键细化则零风险；与第十章 UidState"锁是层的、账是 uid 的"同一分层哲学（第九章 §9.3/第十章 §10.2）。

20. **为 BatteryService 设计"电量数据健康监控"：如何发现 HAL 停更、数据跳变与多源不一致？给出可落地的三个探针与降级策略。**

参考答案要点：探针①心跳：在 update() 里记 lastUpdateTime，watchdog 超过 N 分钟无帧且未处于已知静默（如满电恒温）则 FrameworkStatsLog 告警；探针②一致性：周期对照内核 /sys/class/power_supply 的 capacity/voltage 与 HealthInfo 偏差超阈上报；探针③跳变检测：单帧 level 阶跃超限（如 60→20）记 EventLog 供事后归因。降级：HAL 停更时粘性快照加 EXTRA 停更标记（或沿用 mLastHealthInfo 旧值+超时置 STATUS_UNKNOWN 触发 isPoweredLocked 的"未知=有电"保守分支），避免下游（安全关机/省电触发）吃过期数据（第十一章 §11.2-11.3）。

术语表

术语	解释
ACQUIRE_CAUSES_WAKEUP	WakeLock 标志：拿锁即亮屏；仅屏幕类锁+特权应用或 TURN_SCREEN_ON appops 允许，13 里按 PowerGroup 定向唤醒（PMS:1585-1640）
EJ 配额	加急 Job 的独立配额账本：24h 窗口内 EXEMPTED 45 分钟递减到 RESTRICTED 5 分钟（QcConstants:3170-3175），与常规配额分账分 Timer
HealthInfo	health HAL 上报的电量结构体：level/status/temperature/chargeCounter 等，BatteryService.mHealthInfo 是全系统电量唯一真源
HealthServiceWrapper	BatteryService 与 health HAL 的绑定器：AIDL 优先、NoSuchElement 回退 HIDL，靠服务注册通知处理 HAL 晚起/重启（:109-117）
IN_QUOTA_BUFFER	回款提前量 30 秒：距额度恢复还差 30s 即判"在额度内"，防恢复瞬间满载抖动（QcConstants:3122）
QcConstants	QuotaController 的可配置常量集：窗口/时长/次数/EJ/bump 全在此，可被 DeviceConfig 覆盖（:3105 起）
SuspendBlocker	PMS 持有的 native 拦截对象：wake/display 两类，对应内核 wake_lock/wake_unlock 写入——CPU 挂起资格的最终开关（PMS:3565）
Timer（配额秒表）	QuotaController 内部类（:2101）：每 (userId,package) 一把，记后台执行时长与次数，第一个 Job 开场时作废统计缓存并排 cutoff 断账闹钟
UidState	PMS 按 uid 的人头账：mNumWakeLocks 计锁数、mProcState/mActive 供三种停发判定（acquireWakeLockInternal:1516）
剥夺（mDisabled）	PARTIAL 锁的"停发"：缓存进程/Doze 非白名单/LowPowerStandby 三场景不再计入摘要；锁仍在账本，解除后按需补发 notify（PMS:4017）
长锁哨兵	60 秒检查机制（MIN_LONG_WAKE_CHECK_INTERVAL，PMS:233）：持锁超时的 PARTIAL 锁上报 BatteryStats 标记 long wake lock——dumpsys batterystats 的长锁来源
充电豁免	isQuotaFreeLocked(:903)：充电即免配额（不计功耗成本），唯一例外 RESTRICTED——行为惩罚不随资源状况解封
触发限流	不分桶的突发钳制：过去 1 分钟最多启动 20 个 Job（:3139-3140）；判法"限流窗过期或窗内计数<20"避免逐个滑窗扫描

术语	解释
断账闹钟	Timer.scheduleCutoff(:2286)：按 getTimeUntilQuotaConsumedLocked 推出"再跑 X ms 就超账"的时刻，用 uptime 延迟消息强制断账（Job 在跑则系统必醒，uptime 安全）
关机双闸	BatteryService 的同步安全阀：电量临界未充电（shutdownIfNoPowerLocked:382）与电池温度超 68°C（shutdownIfOverTempLocked:402），直接发 ACTION_REQUEST_SHUTDOWN
会话（TimingSession）	一段连续后台执行的账目单元（起点/终点/Job 数），由 emitSessionLocked(:2191) 在最后一个后台 Job 结束且非充电时落账
会话合并	间隔小于 5 秒（DEFAULT_TIMING_SESSION_COALESCING_DURATION_MS）的相邻执行并算一段——封死"循环短睡刷会话额度"的擦边
记账结算分离	PMS 全书主模式：*NoUpdateLocked 只改账本，统一由 updatePowerStateLocked 六相结算（userActivity/WakeLock/Doze 联动共用）
粘性快照	ACTION_BATTERY_CHANGED 的语义：REGISTERED_ONLY+REPLACE_PENDING+sticky 合成"此刻状态查询"而非事件流，registerReceiver 传 null 即取当前值
判变	processValuesLocked(:495-506)：11 个字段任一变化或 force 才继续广播——电量微变风暴下只放行"有意义的变化"
配额窗口	按桶区分长度的滚动窗口（ACTIVE 10min/WORKING 2h/FREQUENT 8h/RARE 24h）：时长恒为 10 分钟，速率由窗口折算（第九章 §9.1）
桶预算表	QcConstants 兑现的五桶执行预算：时长额度×窗口×次数上限×会话上限四维，RESTRICTED 为 10 个 Job/1 段会话/天
先取后放	updateSuspendBlockerLocked 的不变式：先挂需要的 SuspendBlocker 再放多余的，两步间不存在"无人拦截"的挂起空窗
回款闹钟	InQuotaAlarmQueue(:2891，ALARM_TAG_QUOTA_CHECK)：按 Package 预约"预计回到额度内"的时刻，到点重估并通知 JSS 重排队——超配额 Job 的唯一唤醒通道

排障手册：后台调度与电源问题定位

通用路线：`dumpsys power`/`dumpsys jobscheduler`/`dumpsys deviceidle` 定位状态 → `dumpsys batterystats` 拉时间轴归因 → `logcat` 对账事件 → 回到对应章节读机制。以下五个场景按"症状→定

位命令→根因→修复"展开，每条命令可直接在设备上执行。

场景一：整夜静置耗电异常（WakeLock 泄漏）

症状：手机灭屏放过夜，早晨掉电 20% 以上；`dumpsys batterystats` 显示 Wake lock holding time 某包遥遥领先；机身夜里有温度。

定位命令：`dumpsys power | grep -A 10 "Wake Locks"`（当前持锁现场：tag/uid/持锁时间，tag 即 acquire() 的字符串）；`dumpsys batterystats --charged <pkg>` 看 Wake lock 段的累计持锁时长与 partial 标记，找 "long wake lock"（第十章 60 秒哨兵的报告）；bugreport 对照 kernel wakeup_sources 找内核侧未释放的唤醒源；GC 日志里搜 "WakeLock finalized while still held"（finalize 的 wtf 现场直播，PowerManager.java:2882）。

根因：三类最常见——acquire 后异常路径漏 release（引用计数模式下 acquire/release 不配对，mInternalCount 永不到 0）；无超时的长任务锁（没有用 acquire(timeout) 的 mReleaser 自纠）；系统组件（sync/GPS/网络回调）持锁等待一个永不回来的回调。

修复：短期用 `dumpsys deviceidle whitelist -<pkg>` 反向验证后催修代码：所有 acquire 配对 try/finally；有超时语义的锁一律 acquire(timeout)（客户端自纠，第十章 §10.1 注释②）；长任务用 JobScheduler 替代锁保活（第三章——锁表达"我在跑"，Job 表达"我该跑"）。

场景二：Job 一直不跑（配额卡死或约束卡死）

症状：`dumpsys jobscheduler` 里 Job 长期处于 pending，STARTED-STOPPED 反复或从未 STARTED；开发者日志无 onStartJob。

定位命令：`dumpsys jobscheduler | grep -B 2 -A 12 <pkg>` 看该 Job 的约束位图与每个控制器的 STOPPED-REASON；`dumpsys jobscheduler | grep -A 30 QuotaController` 看 TimingEvents 时间线与每包 ExecutionStats（窗口内时长/次数/会话是否顶格）；`am get-standby-bucket <pkg>` 确认桶；`dumpsys deviceidle` 确认是否深睡压窗。

根因：四层候选——配额三维（时长/次数/会话）任一顶格（第九章 §9.2）；触发限流 1 分钟 20 个（循环提交型代码）；桶太深（RESTRICTED 一天 1 段 10 个）；约束控制器未满足（网络类型不符、不充电等，第三章 §3.2.1 速查表）。

修复：按 STOPPED-REASON 对表下药：配额卡死先查是否"长跑 Job"（执行计费——把 10 分钟长 Job 拆成幂等短 Job，被卡住的等待不扣钱）；验证手法 `am set-standby-bucket <pkg> active`+插电（充电豁免）二选一解除变量；约束问题改 JobInfo 声明而非运行时重试。

场景三：设备"不睡"（灭屏后仍发热掉电）

症状：灭屏静置半小时，`dumpsys power` 显示 mWakefulness=Asleep 但 CPU 仍醒、温度高；或干脆一直 Awake。

定位命令：`dumpsys power | grep -E "mWakefulness|mUserActivitySummary|Display Power"` 看卡在哪一态；`dumpsys power | grep -A 10 "Wake Locks"` 看是否屏幕类锁（SCREEN_BRIGHT/DIM）或 STAY_AWAKE 撑着；`dumpsys batterystats | grep -i wakefulness` 看 Awake 时间轴；`dumpsys deviceidle` 看是否停在 SENSING/IDLE_PENDING（运动刺激反复回 ACTIVE）。

根因：四种典型——某屏幕类 WakeLock 或 FLAG_KEEP_SCREEN_ON 未撤（第六章）；mUserActivitySummary 一直 Bright（有进程持续 userActivity，如反复的传感器/通知唤醒）；Doze 卡在 SENSING（口袋/桌面震动导致显著运动检测不过，第二章 §2.1 注释②）；PowerGroup 某显示组未 ready（sandman 未沉底，第十二章 §12.2）。

修复：锁持有者按 tag 定位（第十章 tag 即代码现场）修代码；`dumpsys deviceidle force-idle deep` 复现深睡路径验证瓶颈是否消失；用 `dumpsys battery unplug` 排除"充电不睡"的开发选项干扰（stay-on-while-plugged）。

场景四：电量显示停更、跳变或与硬件不符

症状：电量数字长时间不动、单帧大幅跳变、或与系统/硬件读数不一致；`dumpsys battery` 的字段可疑。

定位命令：`dumpsys battery` 看当前 HealthInfo 全字段（unplug 冻结态会显示 fake 值——先 `dumpsys battery reset` 恢复再测）；`logcat | grep -i health` 找 HAL 注册/重绑/异常日志（第十一章 §11.1）；perfetto 的 BatteryChargeCounter/BatteryCurrent 轨看 update 频率（update():423 的四个 trace counter）。

根因：health HAL 停更（HAL 进程死且未重绑，mHealthInfo 冻结，第十一章场景推演）；厂商 HAL 上报跳变（chargeCounter 阶跃，framework 原样转发——BatteryService 不做平滑，判变只管广播时机）；调试残留（unplug 后忘了 reset，全系统活在假电量里）。

修复：停更先 `dumpsys battery reset` 排除调试残留，再重启 health HAL 进程（或整机重启）验证重绑自愈路径；上报跳变属 vendor 责任，framework 侧可在系统层加变化阈值（判变思想，第十一章 §11.2 注释⑥）；量产验证务必以 reset 后的数据为准。

场景五：突发任务（加急 Job/EJ）被拒或排队

症状：setExpedited 的 Job 不立即跑；或短时突发的一批 Job 只跑了一部分。

定位命令：`dumpsys jobscheduler | grep -B 2 -A 12 <pkg>` 看 isExpedited 标记与 EJ 判定（shouldTreatAsExpeditedJob 依赖桶/豁免）；`dumpsys jobscheduler | grep -A 30 QuotaController` 的 EJ 段看 24h EJ 窗口用量（EXEMPTED 45min→RESTRICTED 5min，第九章 §9.1 注释尾）。

根因：EJ 配额独立于常规配额且深桶额度极小（RESTRICTED 5 分钟/天）；EJ 资格本身被桶限制（非白名单桶的"加急"会降级为普通 Job）；触发限流对 EJ 一视同仁。

修复：把真正的突发需求留给 FGS 通道（书4 §3.4，5 秒大限内启动）；EJ 适合"马上要但可等几十秒"的任务；配额耗尽等 24h 窗口滚动或 quota bump（8h 窗最多 8 次，每次 +1min/+2 Job/+1 session）；不要用高频 EJ 绕常规配额——两本账都盯得上你。

10.1 电源管理实战：如何在后台任务和省电间取平衡

```text 后台任务的"生存指南"（读码归纳）
你的 App 想在后台干活，按优先级选择方案：
① 立即（用户可见的 UI 同步操作）→ 直接调（App 在前台）
② 5 秒内 → startForegroundService（书4 §三.5 的 5s 大限）
③ 稍后（设备插电/空闲/Wi-Fi）→ JobScheduler（本册第三章）
④ 周期性 → WorkManager periodic（底层 JobScheduler）
⑤ 精确定时（闹钟）→ setExactAndAllowWhileIdle（书9 §四，9min 限频）
⑥ 云推送 → FCM/厂商通道（不占后台预算）
// 讲解注释①：六个方案的功耗从低到高——
// ① < ② < ④ < ③ < ⑥ < ⑤
// 原则：**能用 Job 就不用闹钟，能用 WorkManager 就不用前台服务**
```

10.2 WakeLock 的正确用法（与滥用分析）

```text WakeLock 分类与正确用法
• PARTIAL_WAKE_LOCK：只保 CPU 不挂起（最常用也最危险）
→ 正确用法：绑定操作（如 GPS 定位），用完立即 release
→ 滥用：忘记 release → CPU 永不挂起 → 耗电第一杀手
• SCREEN_BRIGHT/SCREEN_DIM：保亮屏（已被 FLAG_KEEP_SCREEN_ON 替代）
• PROXIMITY_SCREEN_OFF_WAKE_LOCK：接近传感器灭屏（通话时）
// 讲解注释①：dumpsys power 的 WakeLock 汇总是排障第一现场
// 每个 wakelock 都有 tag（acquire 时的字符串）
// → 对应 App 代码中 acquire() 的参数
// → 滥用的现场直播
```

附录 E：后台调度与电源核心机制深度速查表

机制	一句话	源码锚点	所在册
路径新布局	Job/Alarm/Doze 在 frameworks/base/apex/jobscheduler/	apex/jobscheduler	本册
十二控制器	Connectivity/Battery/Idle/Quota/... "与"闸门	controllers/	本册
QuotaController	10min+24h 双窗配额计费	QuotaController:887	本册
bind 执行	JobServiceContext(:79) implements ServiceConnection	JobServiceContext	本册

机制	一句话	源码锚点	所在册
attach 兑现	mPendingServices/mPendingBroadcast/mLaunchingProviders	AMS attachApplicationLocked	本册/册4
Doze 七态	ACTIVE→INACTIVE→PENDING→SENSING→[LOCATING]→IDLE↔MAINTENANCE	DeviceIdleController:161	本册
QUICK_DOZE	SENSING→QUICK_DOZE_DELAY→IDLE（跳过定位旁路）	DeviceIdleController:200	本册
维护窗递减	首个约 5min，后续缩短（同步需求递减假设）	DeviceIdleController	本册
Alarm	setImpl(:2210)→timerfd（内核 alarm 用户态化）	AlarmManagerService	本册
AllowWhileIdle	每 App 每 9 分钟一次深睡豁免	AlarmManagerService	本册
五桶	ACTIVE(10)→WORKING_SET(20)→FREQUENT(30)→RARE(40)→RESTRICTED(45)	AppStandbyController	本册
PowerGroup	13 新：按显示组独立唤醒度（多屏）	PMS:1252	本册
SuspendBlocker	真正拦 CPU 挂起的 native 对象	PMS/SuspendBlocker.java	本册
LowPowerStandby	13 新：息屏外设极简态	LowPowerStandbyController:71	本册
Thermal	HAL 回调→mitigation/亮度帽→:267 shutdownIfNeeded	ThermalManagerService	本册
BatterySaver	策略叠加器（亮度/AOD/频率/后台限制）	power/batterysaver/	本册